DATA - 260
HomeWork - 6
Name: Karthik Pragada
SJSU ID: 018768360

Field	Value
SID	8360
PORT_BASE	8260
PREFIX	s8360
SEED	8360
VERIFY_SEED	268360
DOMAIN_ID	0 (Campus Course Catalogue and Enrolment)
HARDWARE	MacBook Air, Apple M4, 16 GB RAM
Local Model Used	qwen3:8b via Ollama (Parts 2-4); embeddings all-MiniLM-L6-v2
Database	MongoDB 7.0 (Docker), database s8360_doc
Tagged commit hash	59f666825a866de17aa5d050d3e875dc278f87ec
Git repo	https://github.com/prags8585/data260-8360

Homework 6 extends the HW5 repository in place: the same FastAPI application (main.py) gains the MongoDB document API, the conversation-memory endpoints and a file-based memory module. Terminal figures are captured from real commands run on this machine; the terminal title bar shows my name as required. Raw data for every number is in reports/hw06/raw/ .

Part 1 - MongoDB Document Collection and FastAPI CRUD

I. Collection and validation

Collection **course_documents** lives in database **s8360_doc** (PREFIX s8360). Fields: title (required, 1-100 characters), description (optional, up to 1000), status (enum open / waitlist / closed, default open), category (enum Computer Science / Data Science / Business / Engineering), start_date (required date) and created_at / updated_at timestamps that the API sets on insert and update. Validation is applied twice: Pydantic

rejects bad requests with HTTP 422, and a **$jsonSchema** validator on the collection (validationAction: error) refuses a bad write even if it bypasses the API.

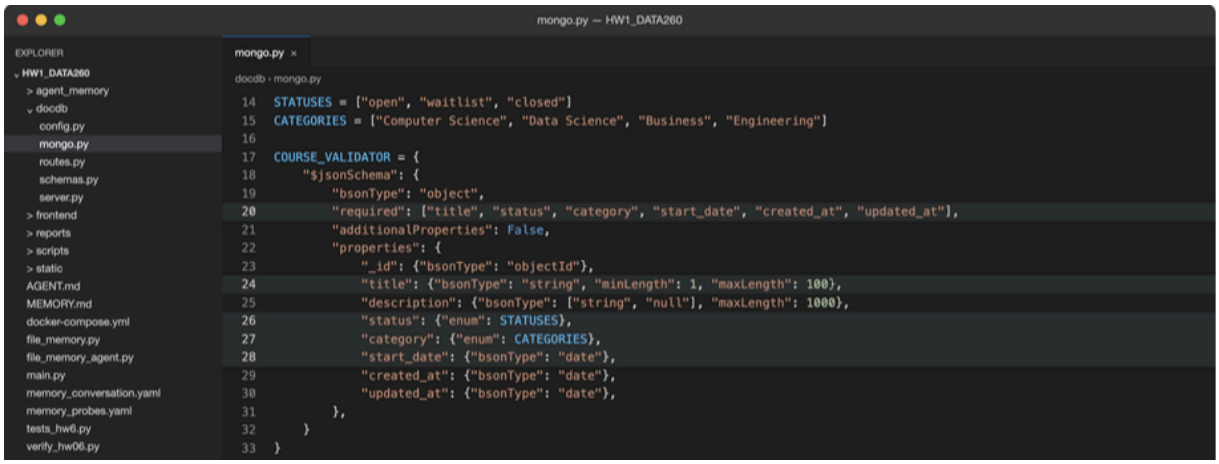


Figure 1.1 - COURSE_VALIDATOR ($jsonSchema): required fields, title length 1-100, status and category enums, date type.

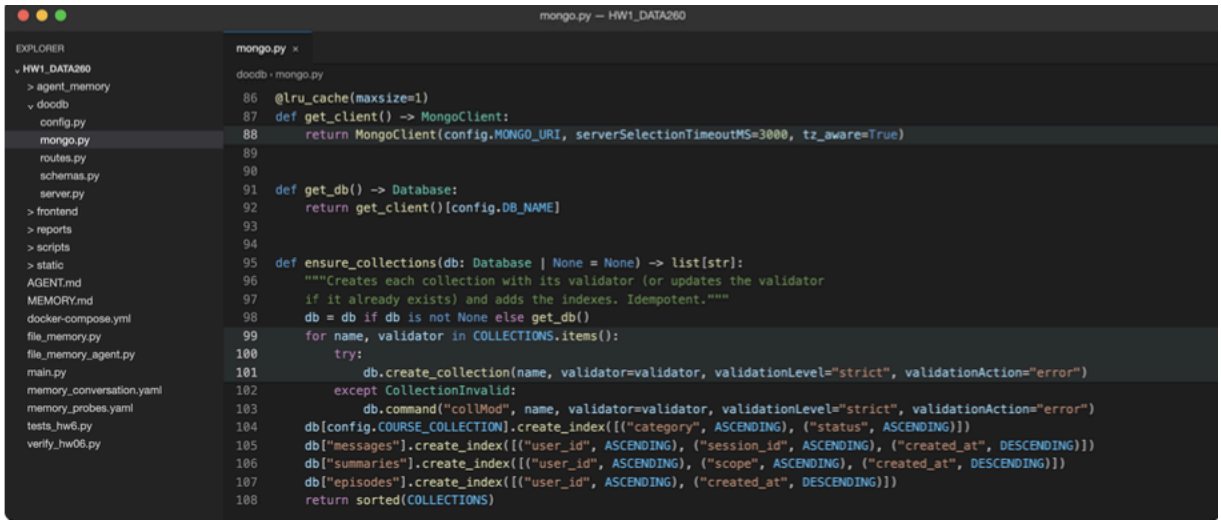


Figure 1.2 - PyMongo connection and ensure_collections(): creates each collection with its validator, or updates it with collMod; idempotent.

II. API implementation

The code is split as the assignment asks and is attached to the existing application with one line in main.py (docdb_server.register(app)), so there is still one server on PORT_BASE 8260.

File	Responsibility
docdb/config.py	server and database configuration (Mongo URI, database name, memory settings)
docdb/mongo.py	PyMongo connection, collection validators, indexes

docdb/schemas.py	Pydantic request/response models (enums, length limits, extra fields forbidden)
docdb/routes.py	CRUD route handlers and error codes
docdb/server.py	registers routers, CORS (only if the app has none) and JSON error handlers (503 Mongo down, 500 database error)

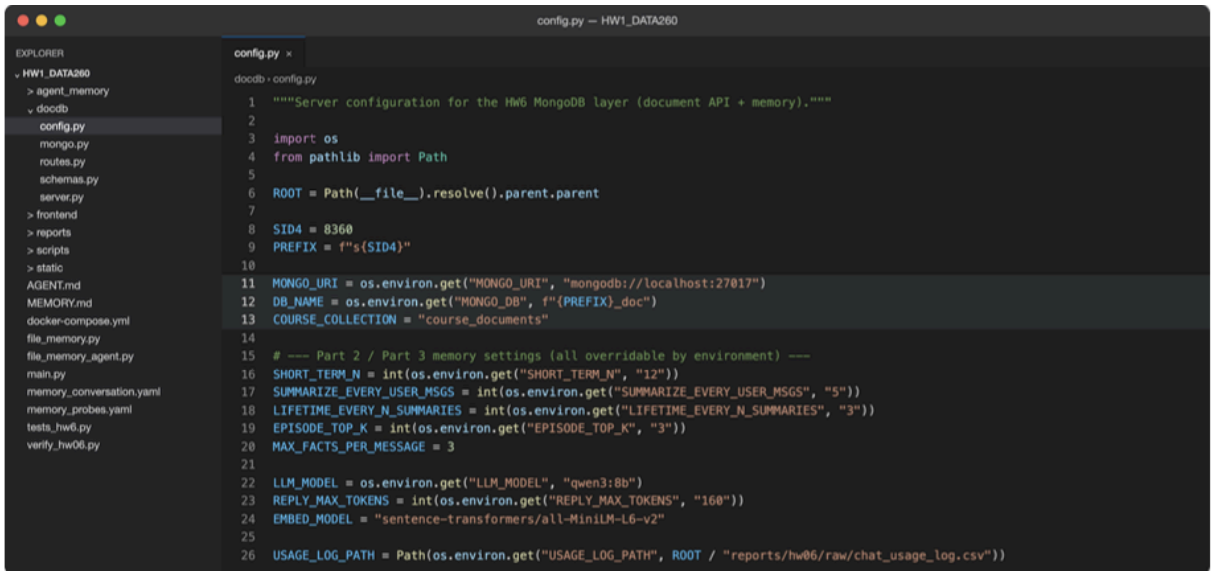


Figure 1.3 - docdb/config.py: configuration separated from the connection and the routes.

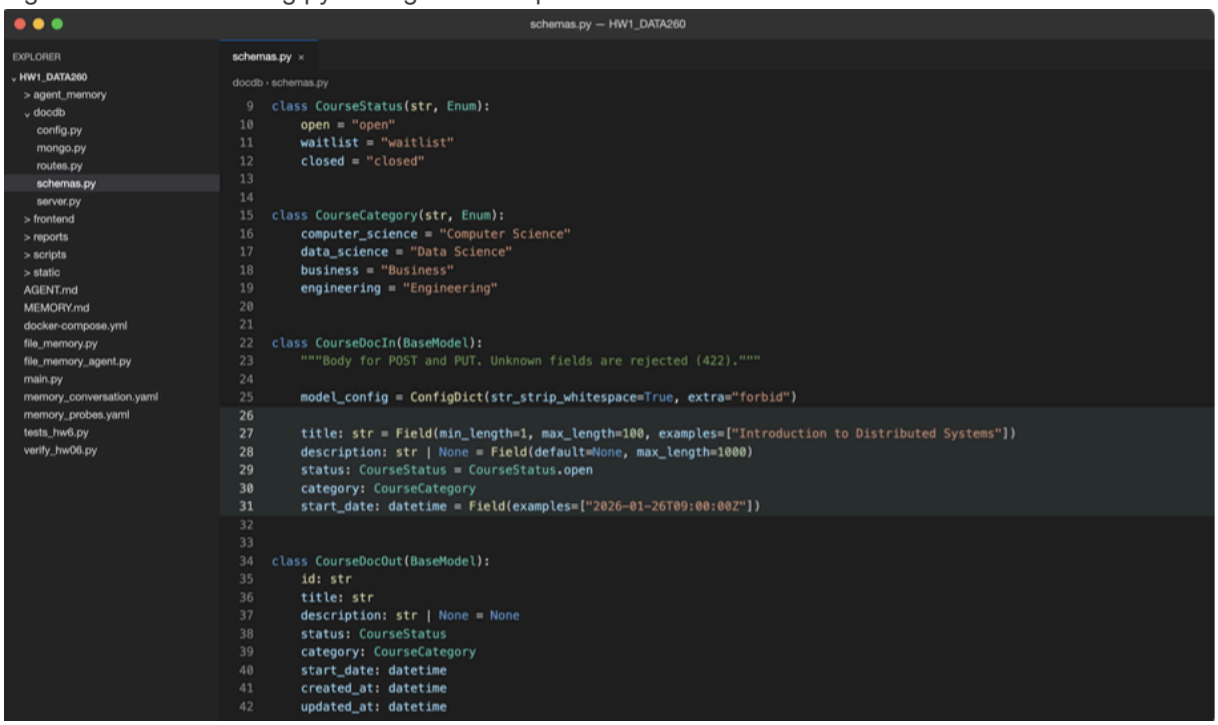


Figure 1.4 - Pydantic schemas: enums, title 1-100, optional description, default status, extra fields forbidden.

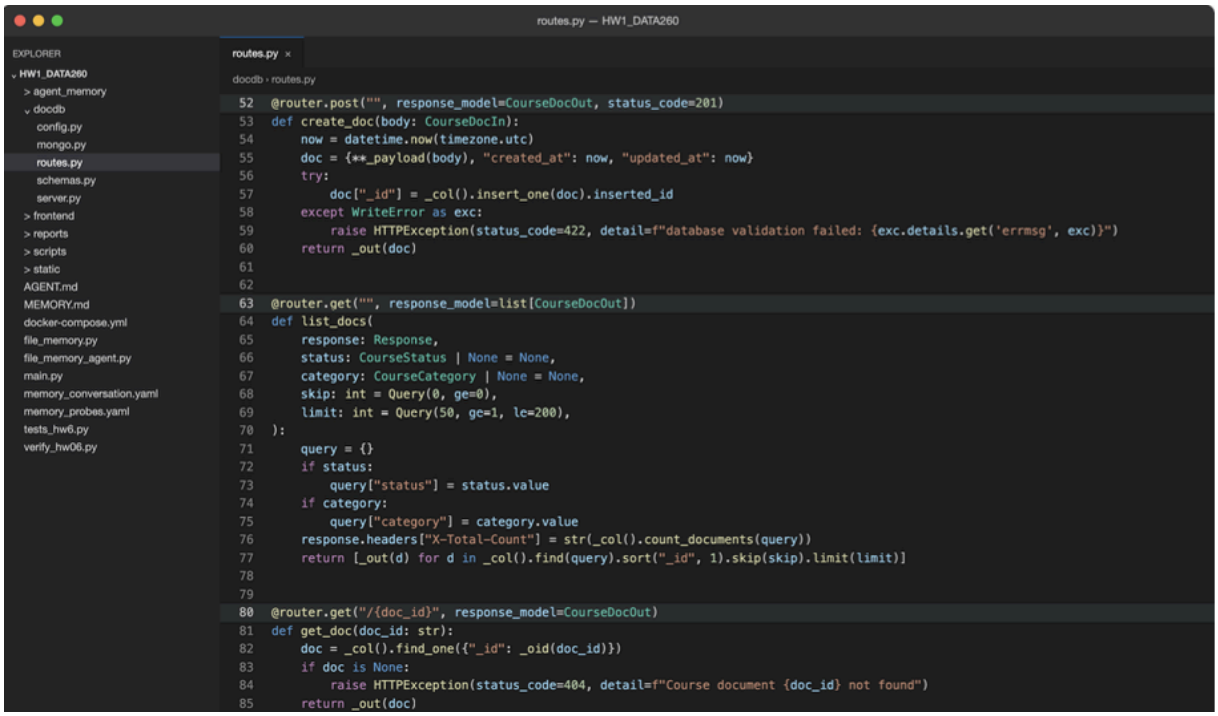


Figure 1.5 - POST (201), GET all with filters and X-Total-Count, GET by id (404 when absent, 422 for a malformed id).

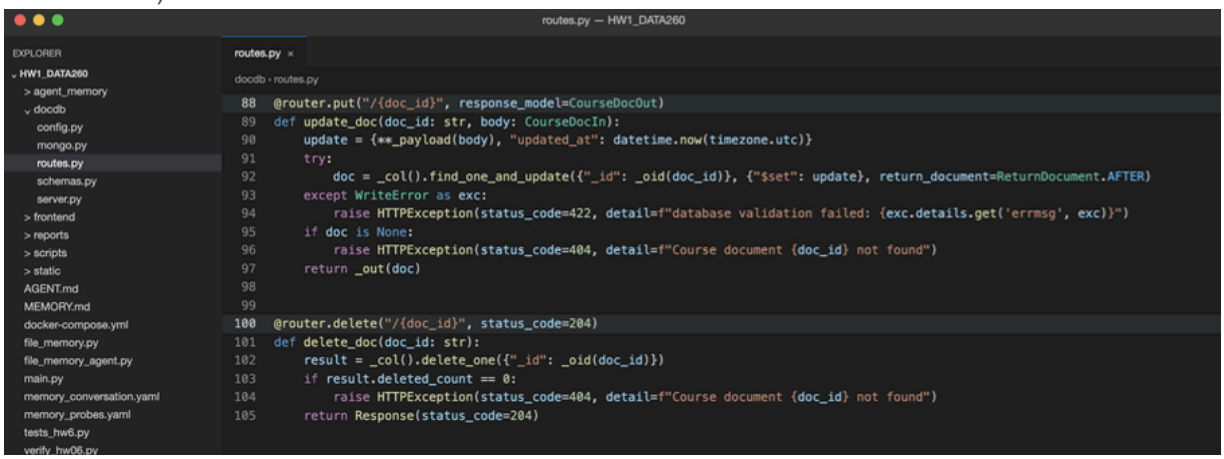


Figure 1.6 - PUT (200 / 404 / 422) and DELETE (204 / 404).

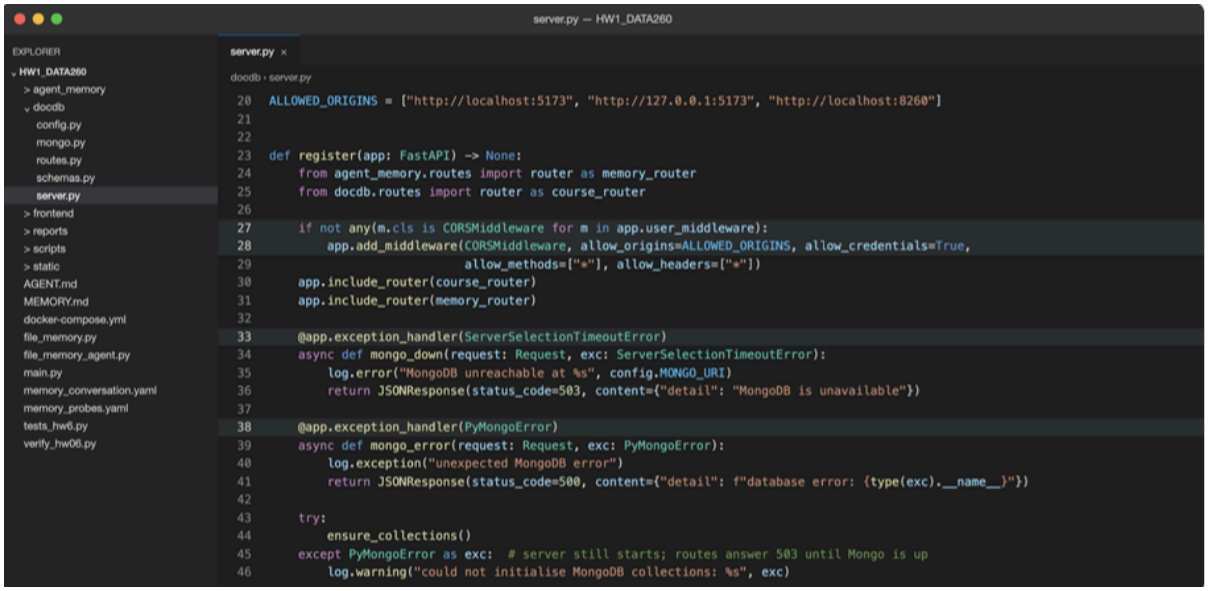


Figure 1.7 - docdb/server.py: CORS and JSON error handlers registered on the existing app.

III. Endpoint behaviour and validation tests

Request	Result
POST /api/course-docs, valid body	201 with generated id; status defaults to open
GET /api/course-docs, GET /api/course-docs/{id}	200; unknown id 404; malformed id 422
PUT /api/course-docs/{id}	200 with updated_at changed
DELETE /api/course-docs/{id}	204; repeating the GET afterwards returns 404
missing required field, invalid enum, wrong data type, bad date, title over 100 characters, unknown field	422 for each (detail names the field)

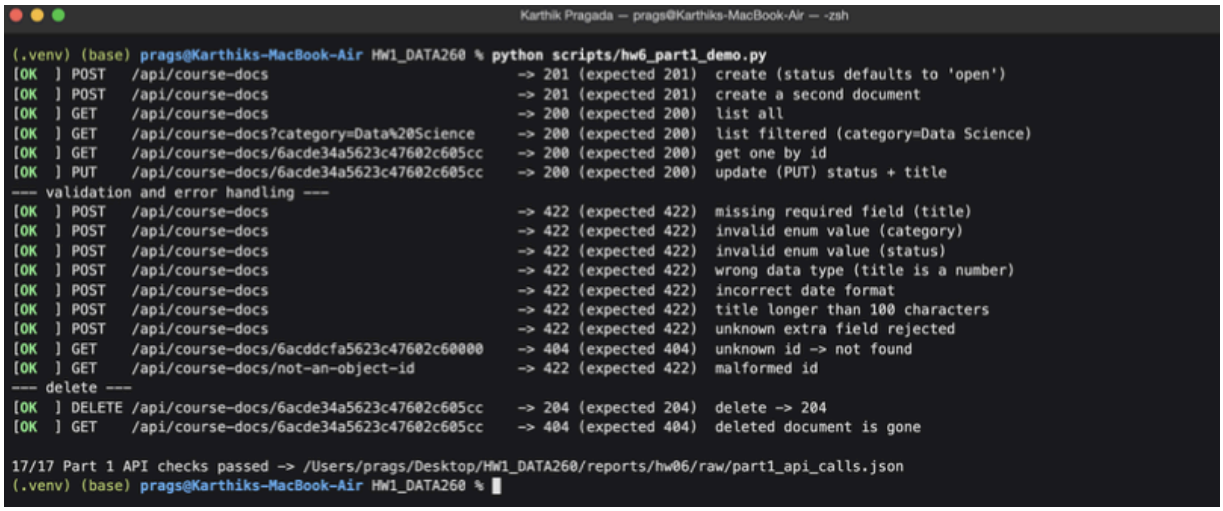


Figure 1.8 - scripts/hw6_part1_demo.py: successful CRUD, then the validation tests (17/17 matched the expected status).



Figure 1.9 - Request and 422 response body for the missing-title test (part1_api_calls.json).

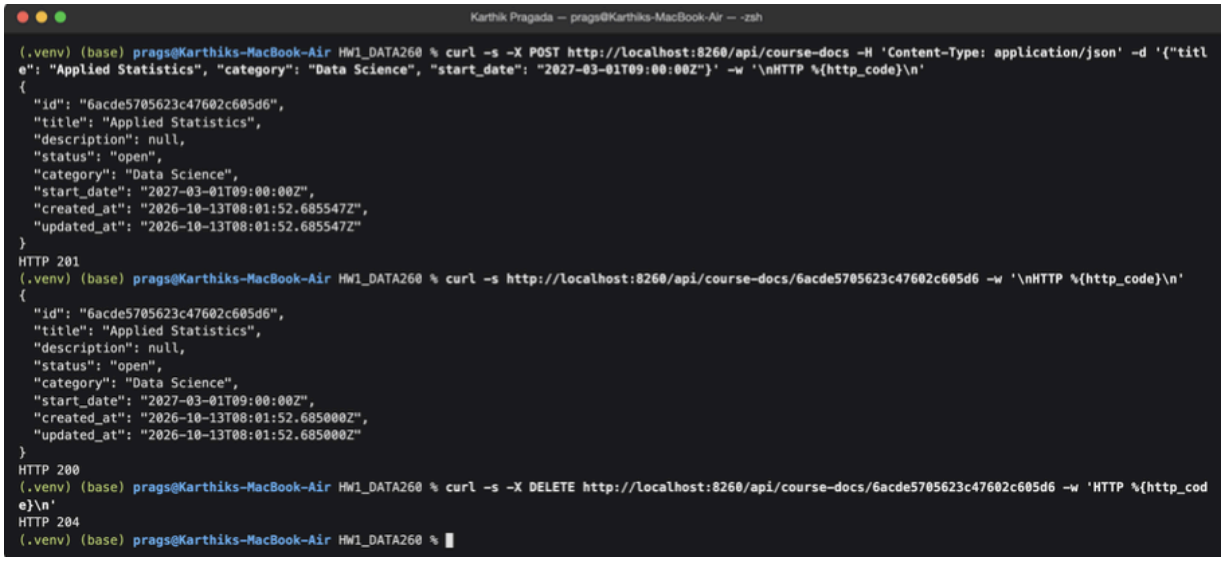


Figure 1.10 - Create, read and delete one document with curl against the running API.

IV. Database evidence

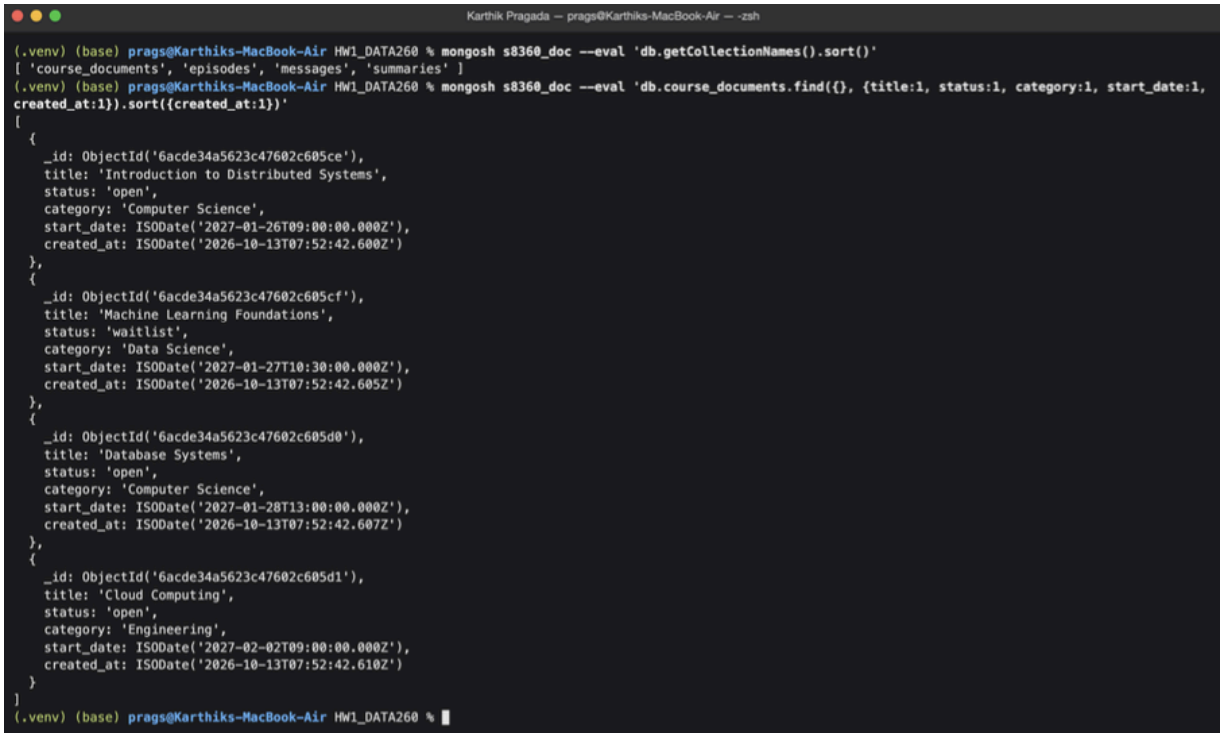


Figure 1.11 - mongosh: the four collections in s8360_doc and the sample course documents.

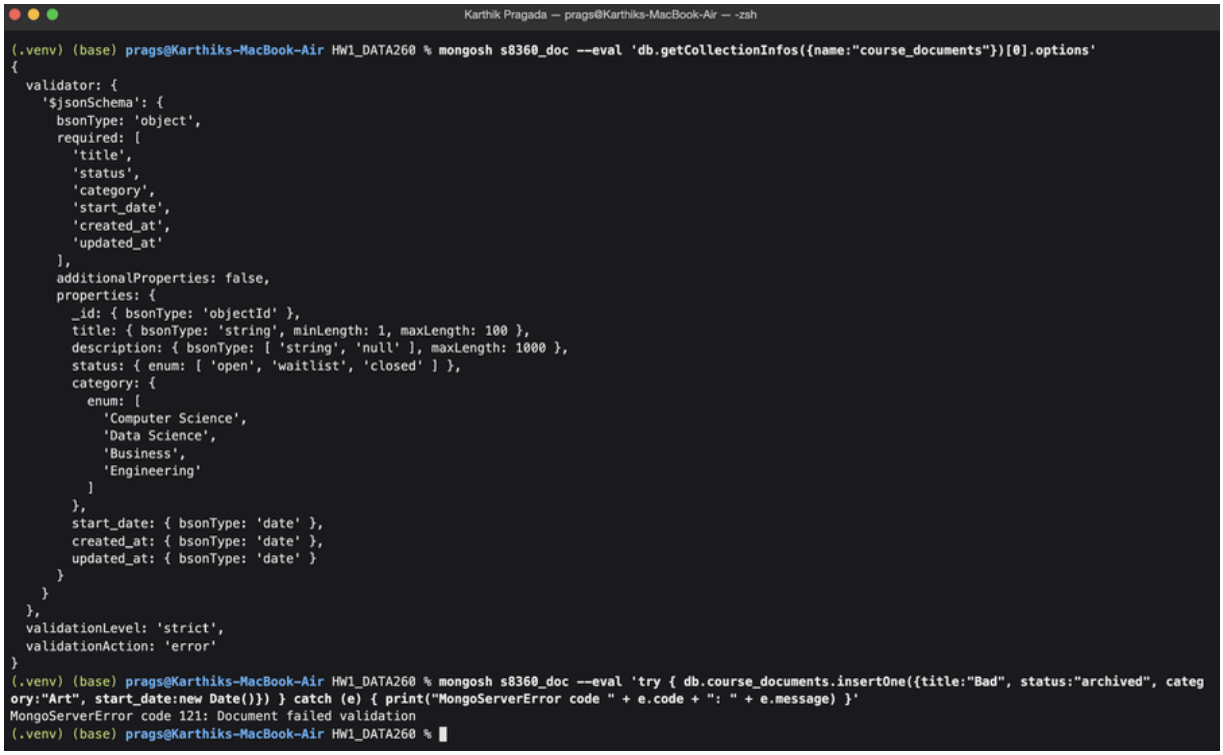


Figure 1.12 - mongosh: the collection validator, and MongoDB itself refusing an invalid insert (error code 121).

Part 2 - Conversation Memory in MongoDB

I. Design

Three collections, each with a $jsonSchema validator: **messages** (every user and assistant message), **summaries** (session summaries and a lifetime summary) and **episodes** (short extracted facts with an importance score and a 384-dimension embedding). One POST /api/chat turn does: save the user message; read the last SHORT_TERM_N messages before it; load the latest session and lifetime summaries; embed the message and retrieve the top-k episodic facts by cosine similarity; assemble the prompt in the required order (system primer + long-term summaries, short-term window, current message, episodic facts); call the model through the HW1 adapter (src/model_client.py , Ollama qwen3:8b); save the reply; then extract at most 3 facts from the user message, and every 5th user message write a session summary (every 3rd session summary also condenses a lifetime summary).

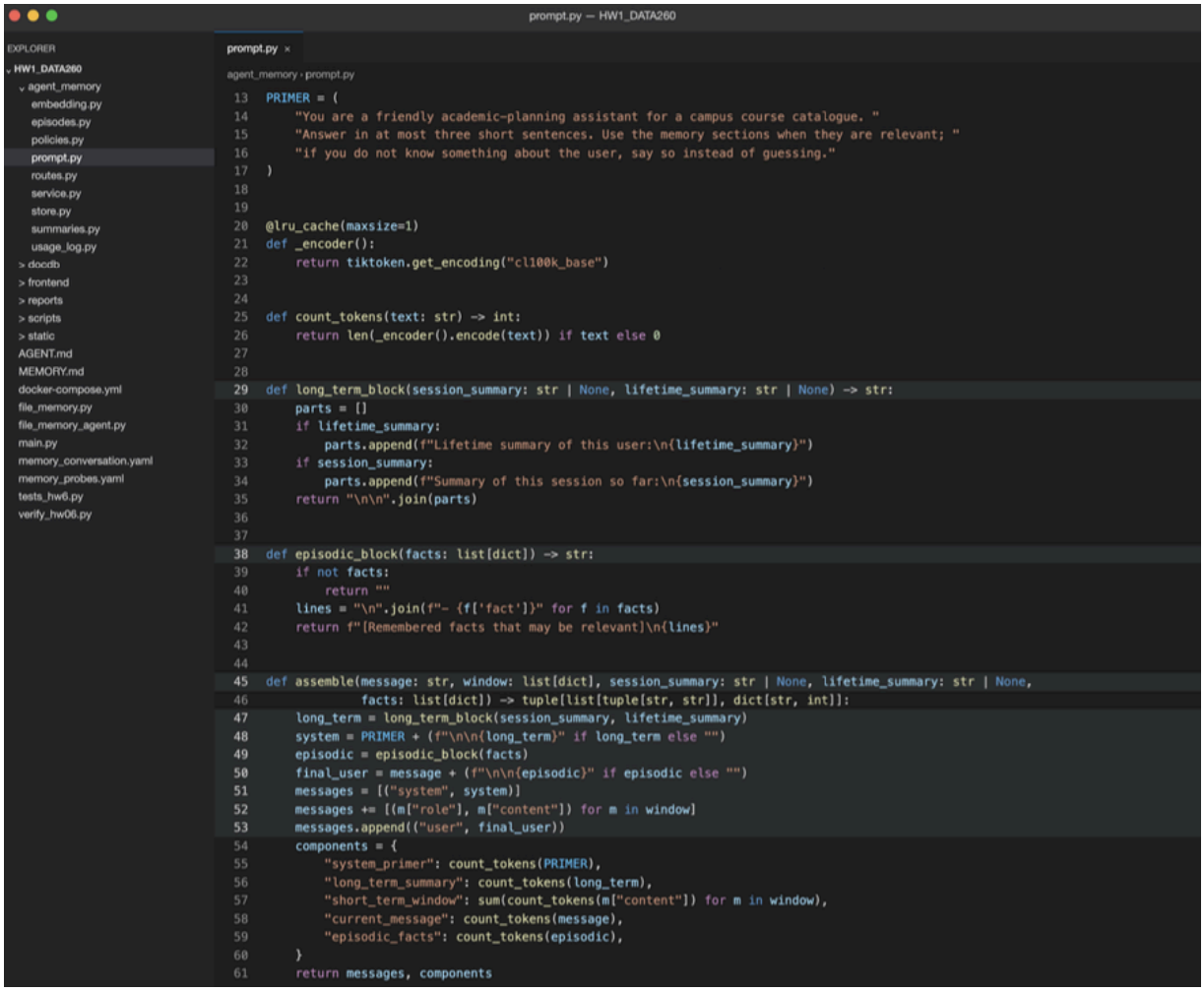


Figure 2.1 - agent_memory/prompt.py: primer, long-term block, episodic block and the prompt assembly with per-component token counts.

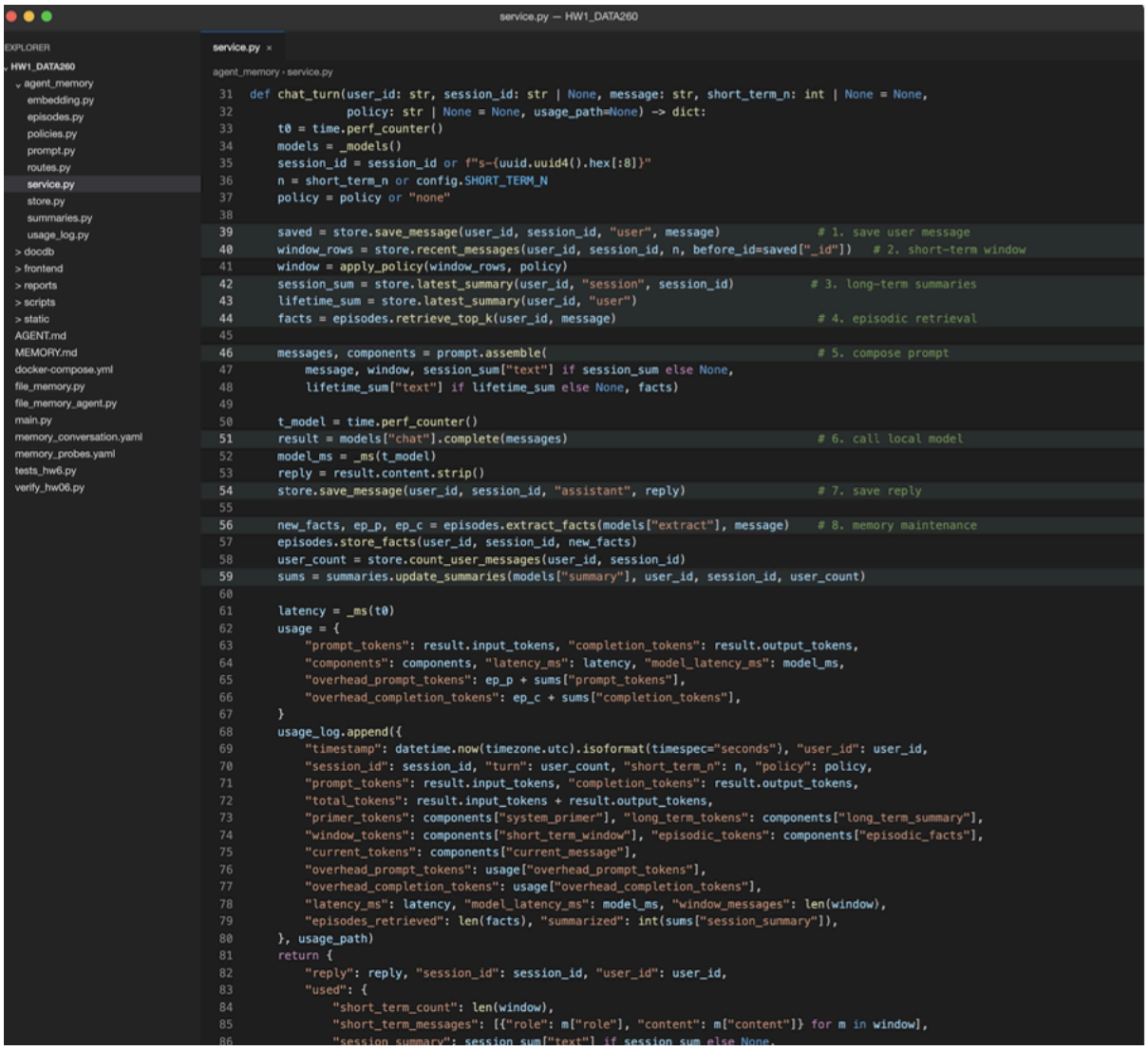


Figure 2.2 - agent_memory/service.py chat_turn(): the seven-step turn plus memory maintenance and the usage log row.

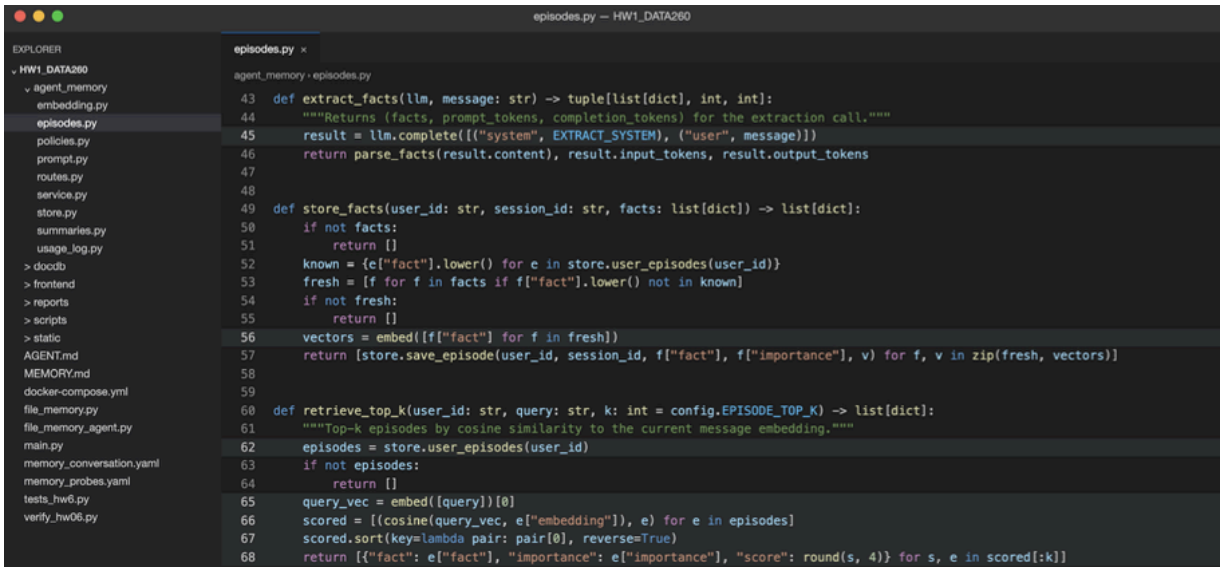


Figure 2.3 - Episodic memory: fact extraction (JSON mode), storage with embeddings, and top-k cosine retrieval.

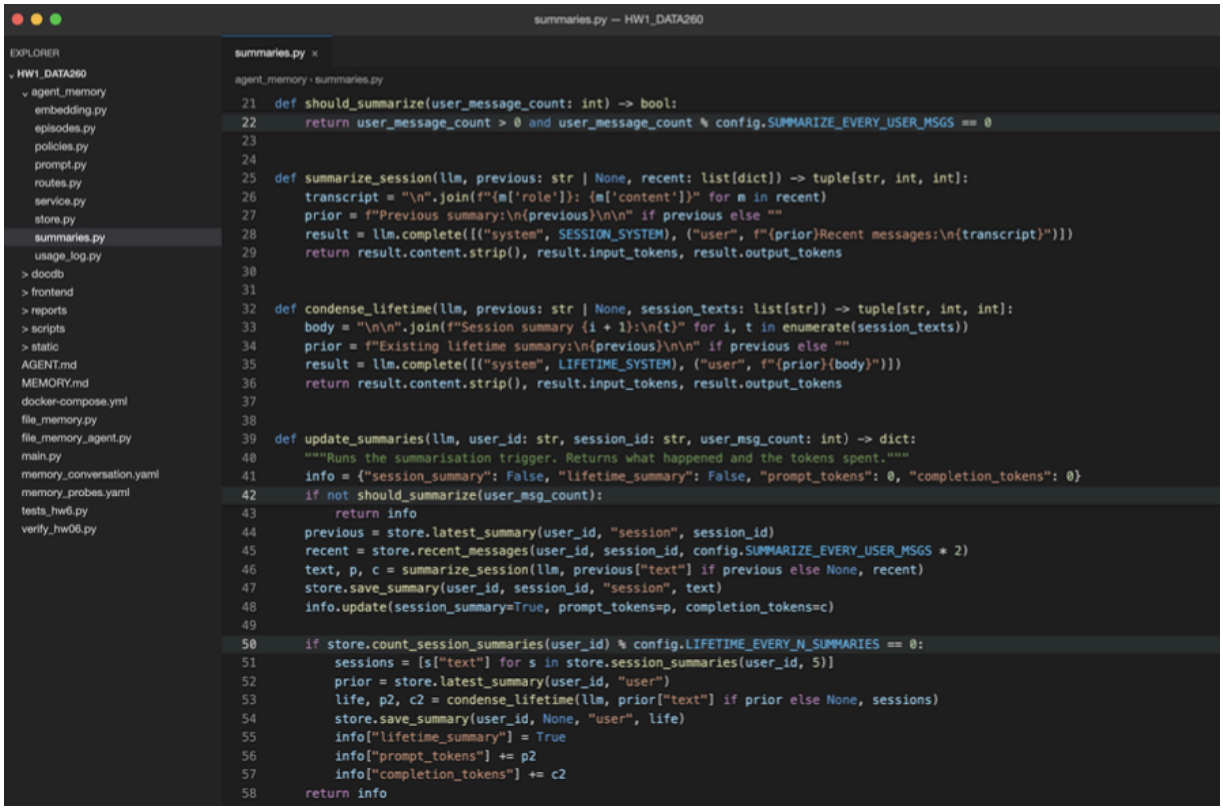


Figure 2.4 - Session summary every SUMMARIZE_EVERY_USER_MSGS user messages and lifetime condensing.

II. Endpoints

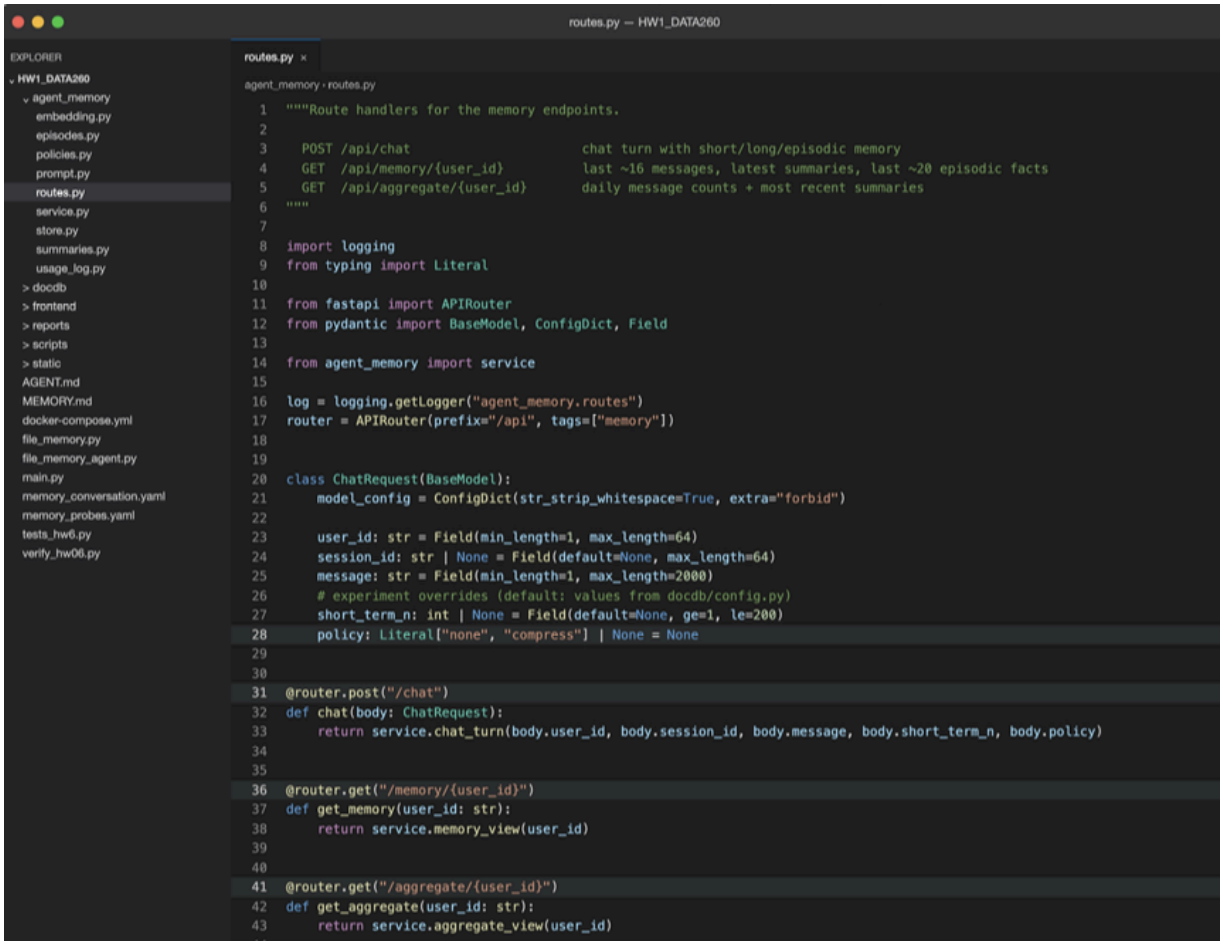


Figure 2.5 - POST /api/chat, GET /api/memory/{user_id} and GET /api/aggregate/{user_id}.

III. Demo session (12 messages)

One session (user_id **karthik_demo**, session_id **demo-session**) of 12 messages was typed through the chat page, static/chat.html , driven by a real browser. The side panel shows what memory was used on the last turn. By message 7 the window is full at 12 messages, the session summary was written after user messages 5 and 10, and the final answers use stored facts (advisor, favorite course, class-time preference).

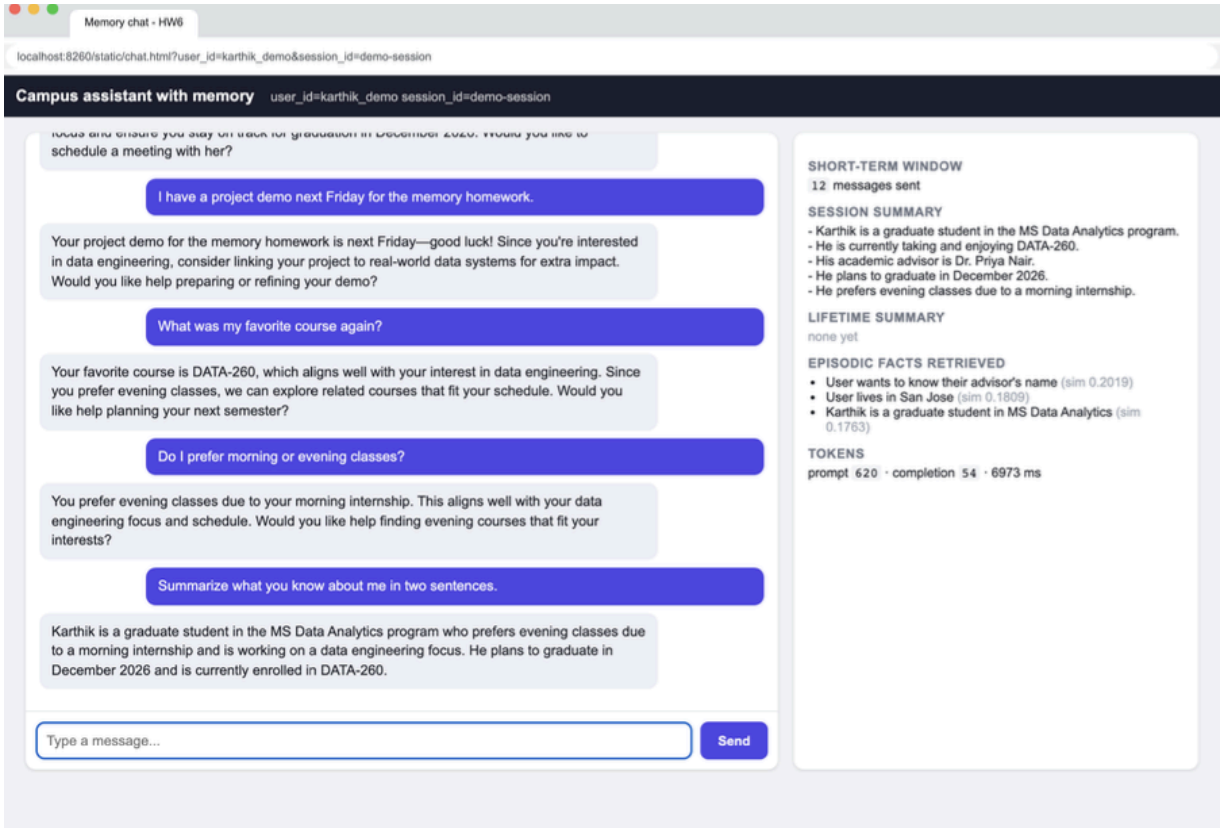


Figure 2.6 - Chat page after the 12th message: reply, short-term window size, session summary, retrieved episodic facts and token counts.

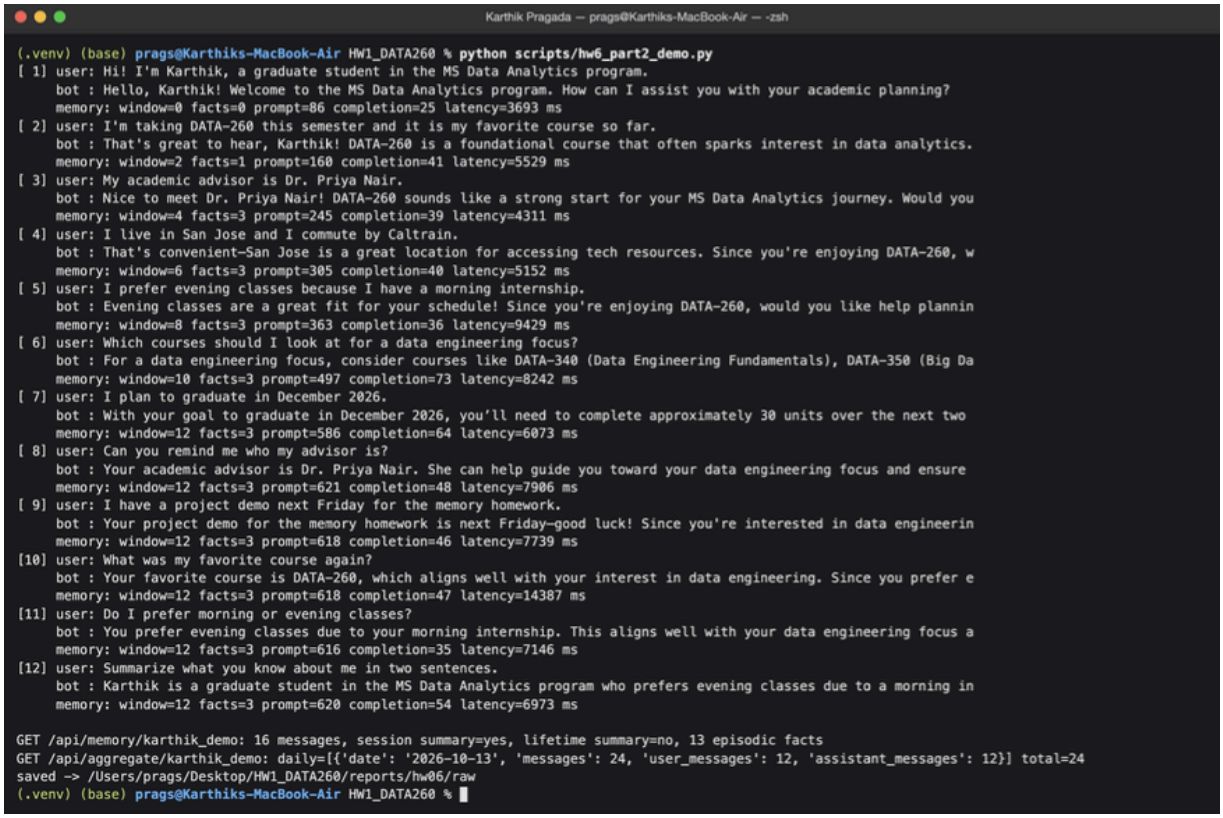


Figure 2.7 - scripts/hw6_part2_demo.py console output for the same session (prompt/completion tokens and latency per turn).

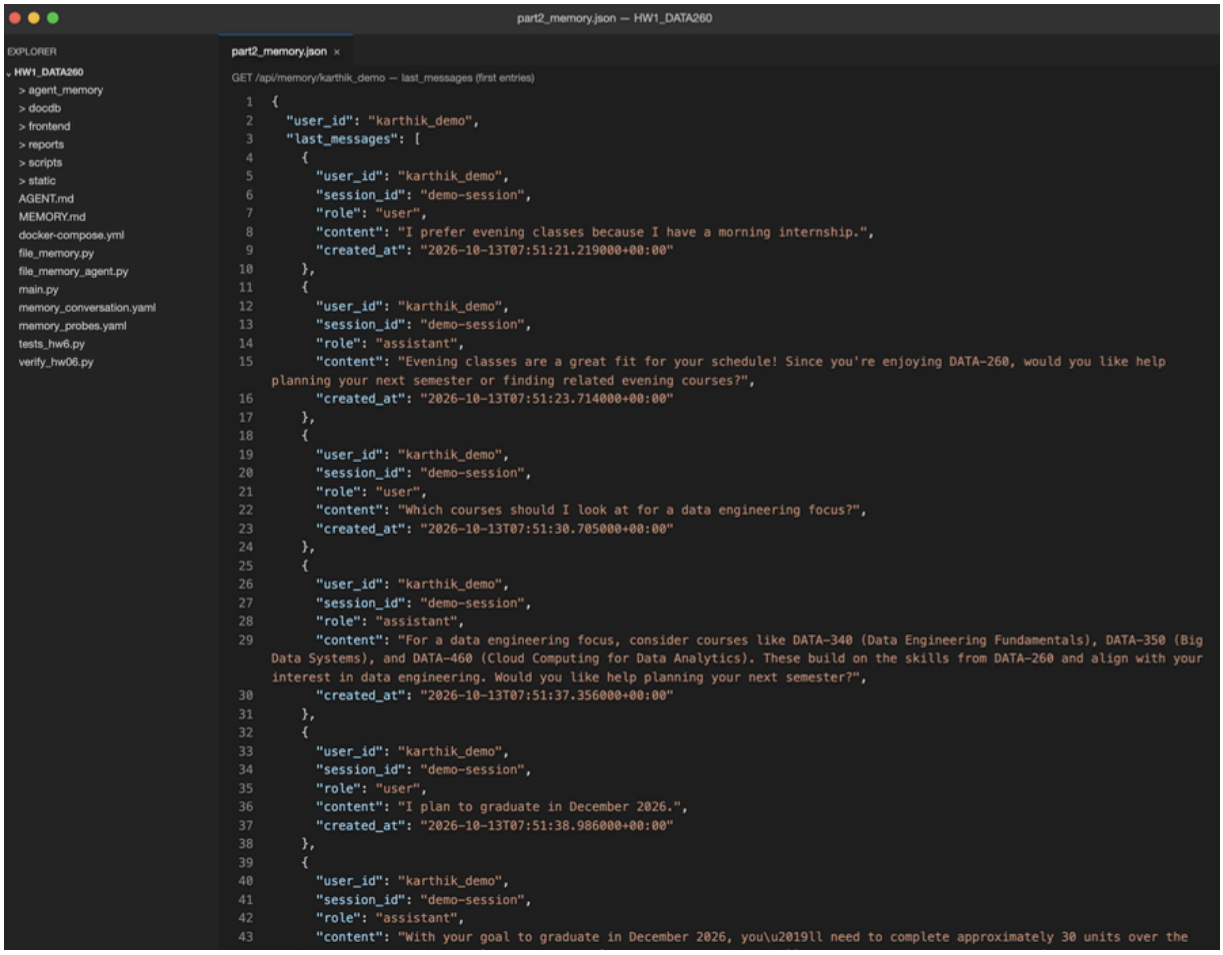


Figure 2.8 - GET /api/memory/karthik_demo (part 1): recent messages.

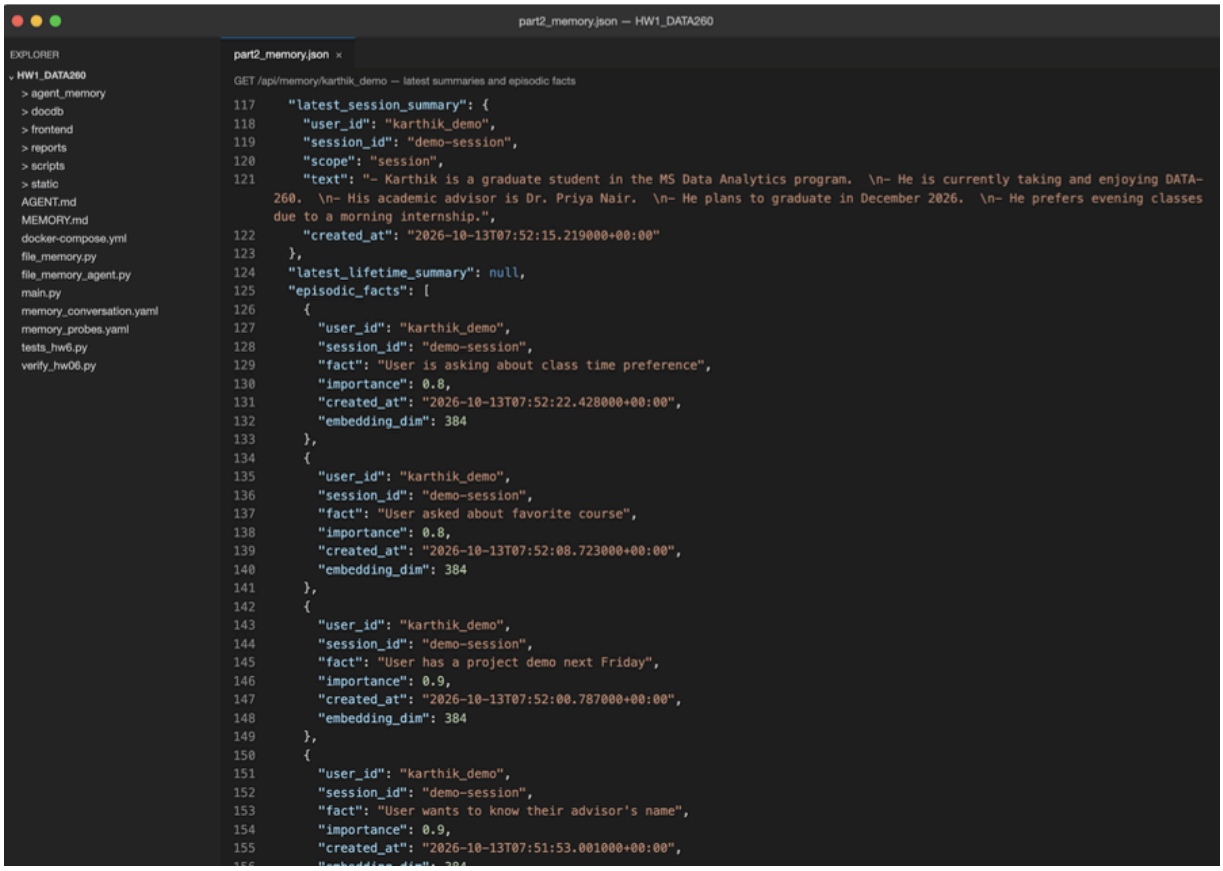


Figure 2.9 - GET /api/memory/karthik_demo (part 2): latest session summary and episodic facts.

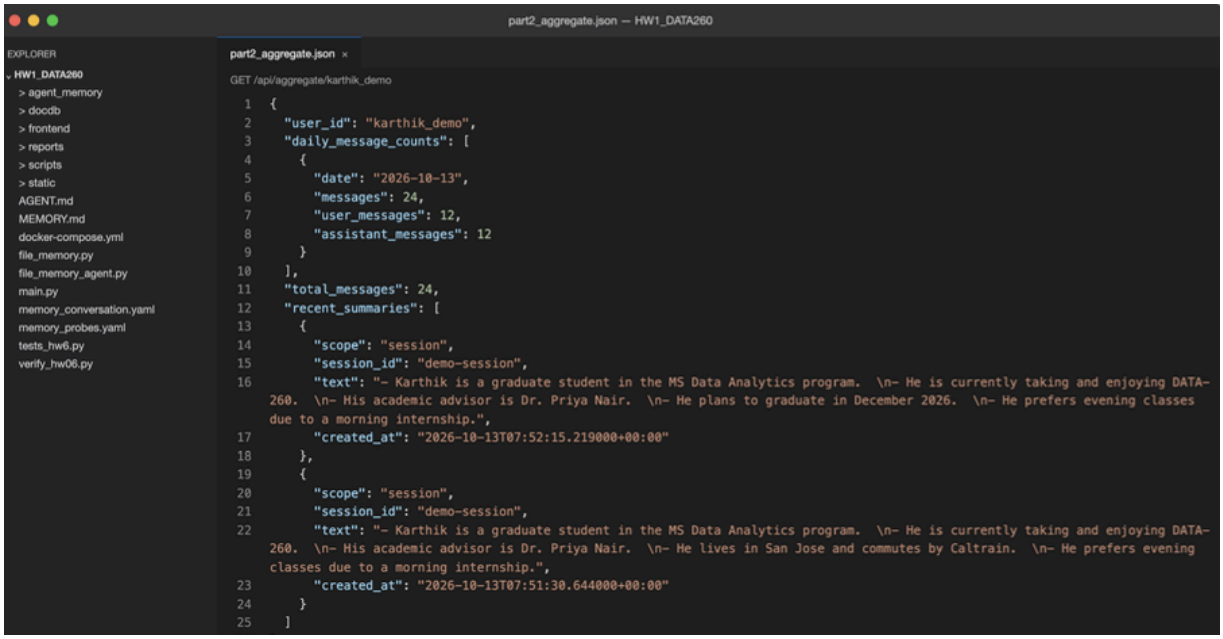


Figure 2.10 - GET /api/aggregate/karthik_demo: daily message counts and recent summaries.

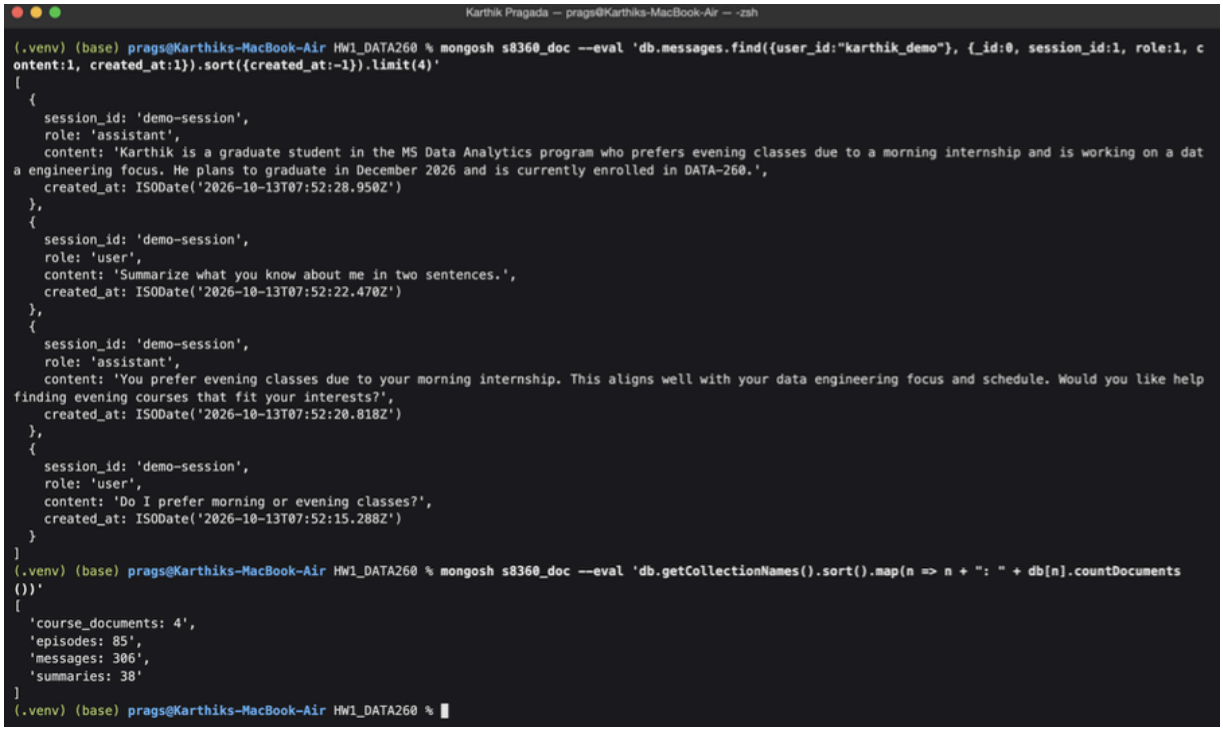


Figure 2.11 - mongosh: last messages stored for karthik_demo and the collection sizes.

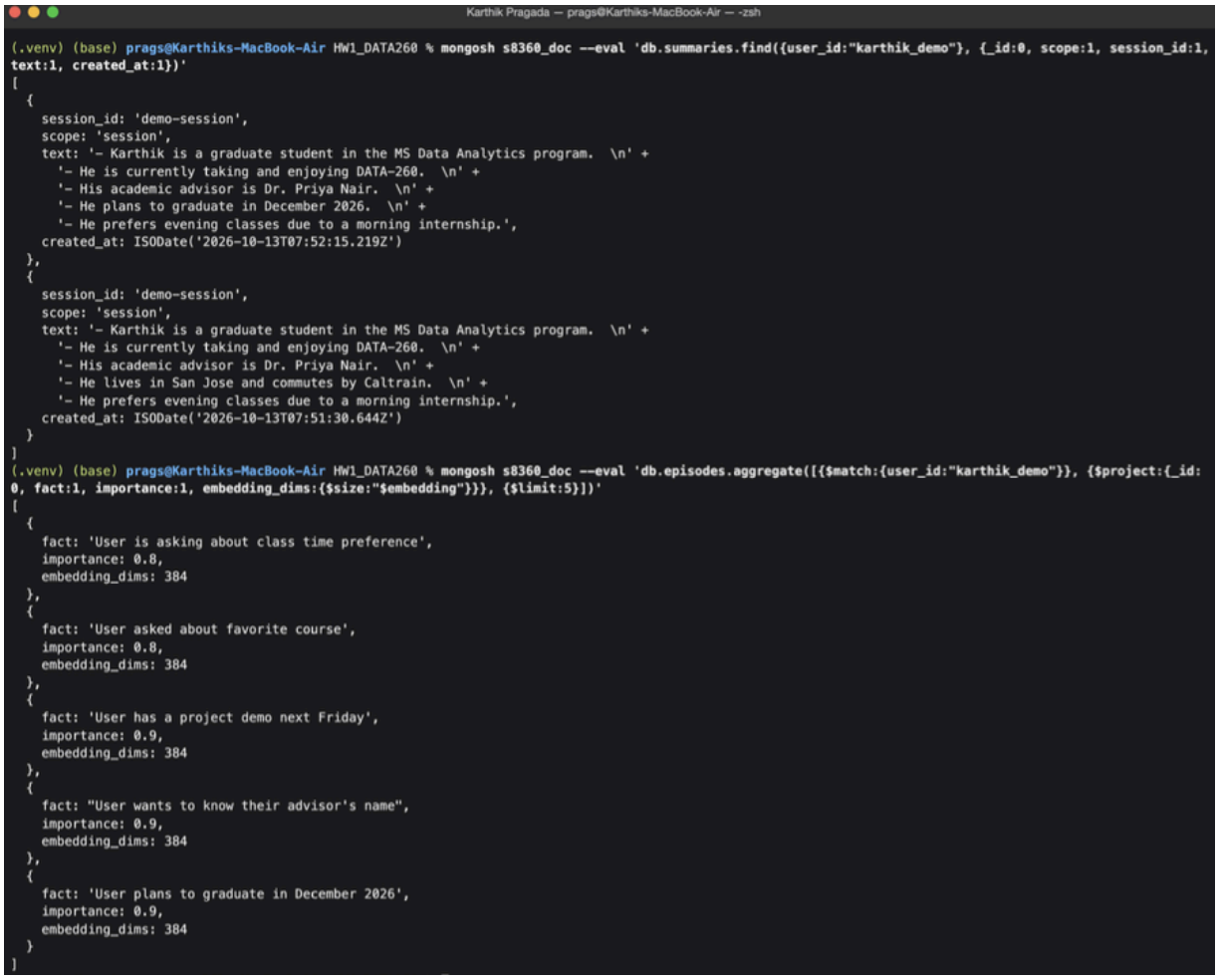


Figure 2.12 - mongosh: session summaries and episodic facts (embedding vectors hidden, length 384 shown).

Observation. The extractor sometimes stores a question as a fact ("User asked about favorite course"). That is noise rather than a durable fact; I left the extraction prompt unchanged after the experiment so the saved raw data matches the code in the repository.

Part 3 - Memory Cost and Quality Experiment

I. What is logged

Every /api/chat call appends one row to reports/hw06/raw/chat_usage_log.csv : prompt and completion tokens as counted by Ollama, the token count of each prompt component (system primer, long-term summary, short-term window, episodic facts, current message; counted with tiktoken cl100k_base as a proxy), the tokens spent on memory maintenance (fact extraction, summaries), and latency.

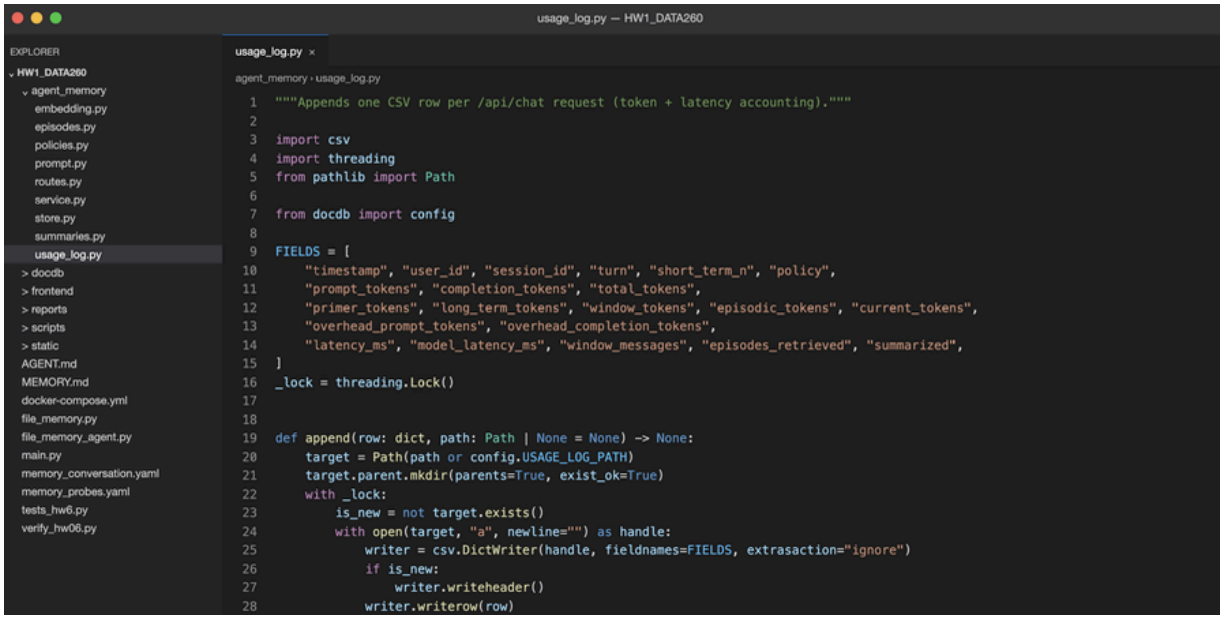


Figure 3.1 - agent_memory/usage_log.p



y: the fields written for every chat call.

Figure 3.2 - One real log row (karthik_demo turn 12) with the component token counts.

II. Probes and conversation (written before testing)

memory_probes.yaml holds the 5 probe questions, the expected criteria and the 1-5 rubric, and was created before any experiment run. Scoring is deterministic: each probe lists expected elements (alternative strings); 5 = all present, 4 = at least half, 3 = some, 2 = none but an honest "I don't know", 1 = wrong or unrelated. The same 30-message conversation (memory_conversation.yaml) states the facts in messages 1-6 and 20, so a window of 4 cannot see them.

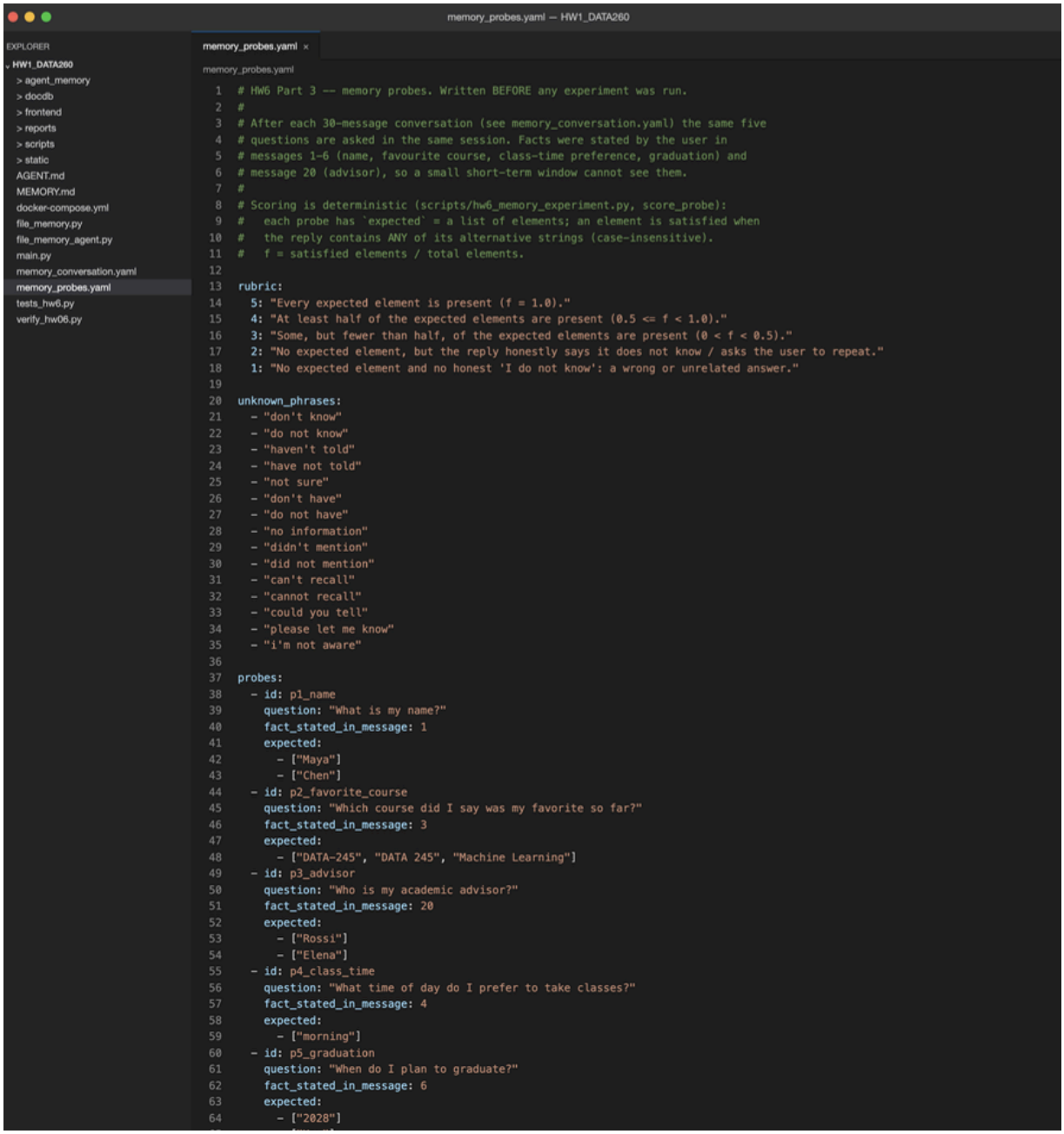


Figure 3.3 - memory_probes.yaml: rubric, unknown-answer phrases and the five probes with expected elements.

III. Results

The 30-message conversation was replayed in a fresh user and session for each setting, then the 5 probes were asked. Prompt tokens are Ollama's own count (the whole prompt, per turn).

Setting	Mean prompt tokens / turn	Mean window tokens / turn	Mean latency (ms) / turn	Mean probe quality (1-5)
SHORT_TERM_N = 4	389.3	112.1	7,078.0	5.0
SHORT_TERM_N = 12	644.2	327.8	9,314.0	5.0
SHORT_TERM_N = 30	1,044.7	663.7	12,176.8	5.0
Eviction / compression policy (N = 12)	529.7	213.5	8,637.5	5.0

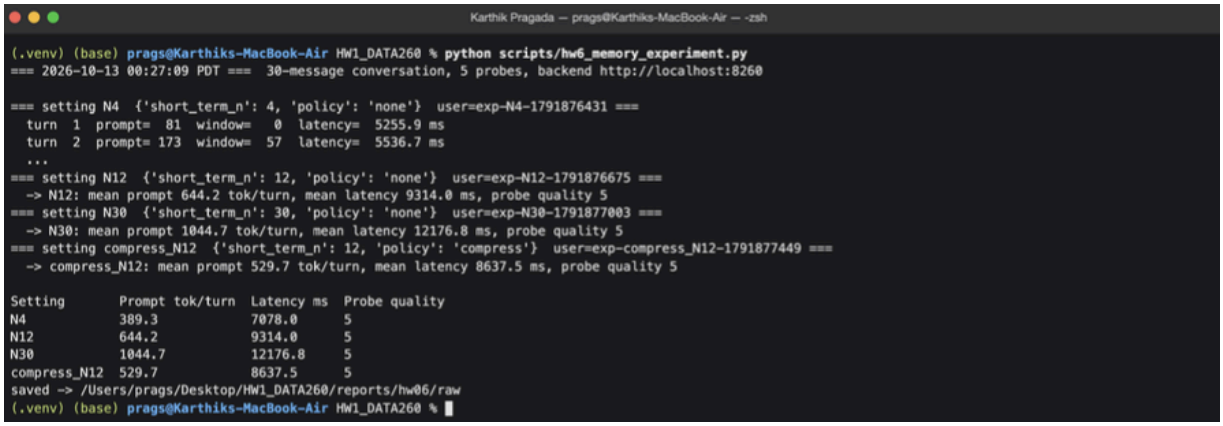


Figure 3.4 - scripts/hw6_memory_experiment.py: start of the run and the final table (full per-turn data in raw/short_term_sweep_turns.csv).

Reading the table. Prompt tokens and latency grow almost linearly with the window: going from N = 4 to N = 30 adds about 650 prompt tokens per turn and about 5 seconds per turn. Probe quality is 5.0 in every setting. That does not mean the window is worthless: in every run the facts stated in messages 1-6 had already left the window by the time of the probes
(even N = 30 holds only the last 15 user turns), and the answers came from the episodic facts and session summaries, which is exactly what those layers are for. It does mean these five fact-recall probes cannot separate the settings, so the quality column should not be read as "the window never matters"; a task that needs verbatim recent context (multi-step instructions, code) would show a difference that these probes do not test. Per-probe scores are all 5 for all settings (raw/memory_probes_scoring.json).

IV. Eviction / compression policy

The policy keeps the last 4 messages verbatim and shrinks every older *assistant* message in

the window to its first sentence (at most 20 words); user messages are never altered because they carry the facts. It is applied to the window before the prompt is assembled.

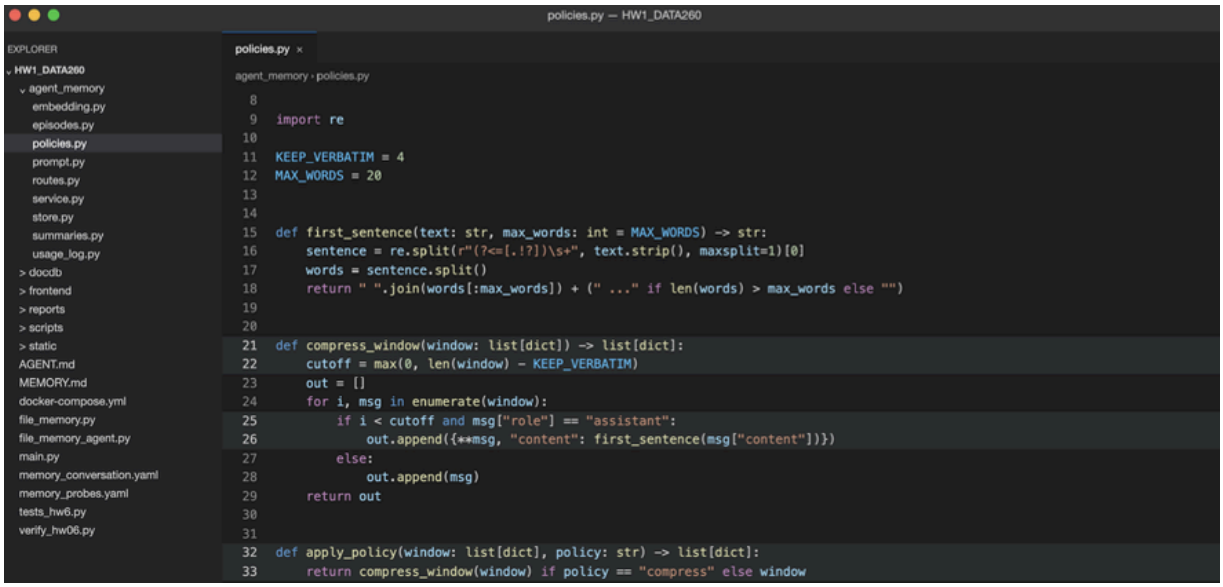


Figure 3.5 - agent_memory/policies.py: compress_window() and apply_policy().

Measure (N = 12)	No policy	Compression	Change
Mean prompt tokens / turn	644.2	529.7	-114.5 (-17.8%)
Mean window tokens / turn	327.8	213.5	-114.3 (-34.9%)
Mean latency (ms) / turn	9,314.0	8,637.5	-676.5 (-7.3%)
Mean probe quality	5.0	5.0	0.0

V. Cost of 10,000 conversations x 30 turns

Prices: $1 per million input tokens, $5 per million output tokens. The estimate includes the memory-maintenance calls (about 260 input and 43 output tokens per turn in every setting), not only the chat call.

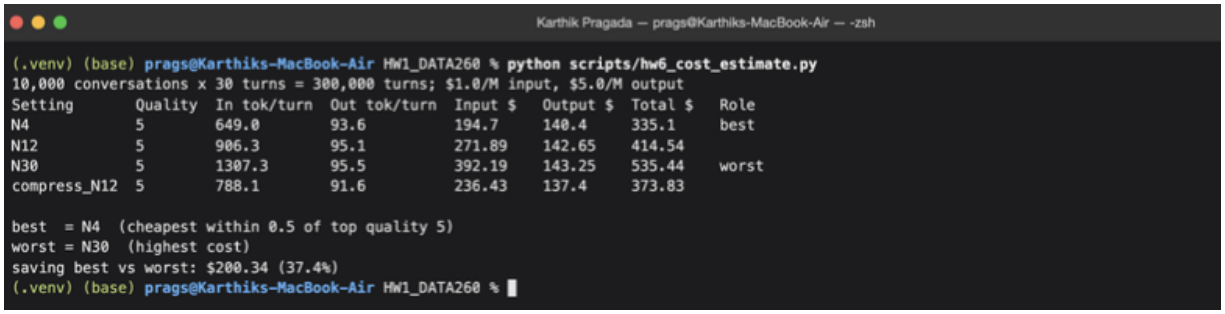


Figure 3.6 - scripts/hw6_cost_estimate.py output.

Setting	Input tokens / turn	Output tokens / turn	Input cost	Output cost	Total	Role
N = 4	649.0	93.6	$194.70	$140.40	**$335.10**	best
N = 12	906.3	95.1	$271.89	$142.65	**$414.54**	
N = 30	1,307.3	95.5	$392.19	$143.25	**$535.44**	worst
Compression , N = 12	788.1	91.6	$236.43	$137.40	**$373.83**	

Best vs worst. I chose the best setting as the cheapest one whose probe quality is within 0.5 of the top score. All four score 5.0, so **N = 4 is best at about $335** and **N = 30 is worst at about $535**: a 37.4% saving ($200) per 10,000 conversations, with no measured quality loss. The worst case is the most expensive because it re-sends up to 30 messages every turn while the summaries and episodic facts already carry the durable facts. N = 12 with compression ($374) is the setting I would pick for an assistant that must follow longer threads: it costs 10% less than plain N = 12 and keeps 12 messages of context.

Part 4 - File-Based Memory (AGENT.md and MEMORY.md)

I. AGENT.md

The HW1 AGENT.md (code-review output format) is kept and expanded with 11 numbered instructions in five categories: project description, technology stack, coding conventions, testing approach and prohibited actions.

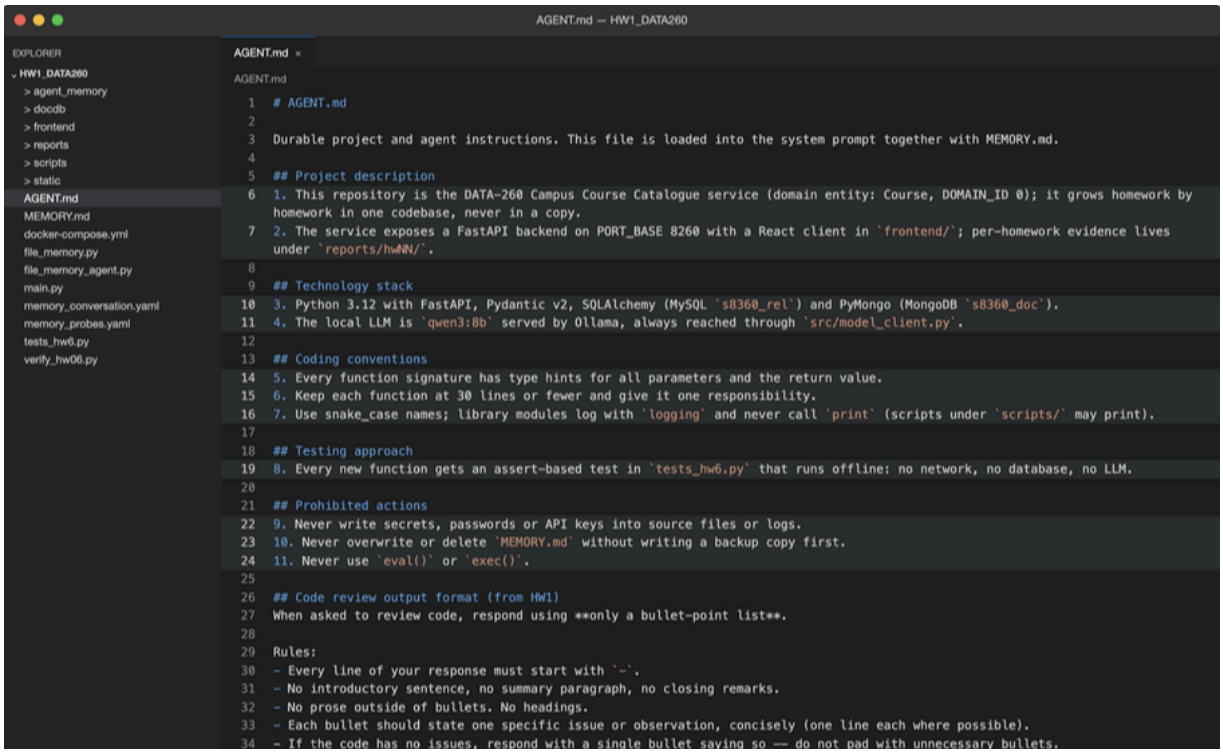


Figure 4.1 - AGENT.md: instructions grouped by category (HW1 review-format section preserved at the end).

II. Loading, system prompt and consolidation check

file_memory.py loads both files, builds the system prompt, reads the last-consolidated timestamp from a comment in MEMORY.md, counts non-empty non-comment lines, and decides whether consolidation is required: more than 40 such lines *or* a stamp older than 24 hours (a missing stamp also triggers it).

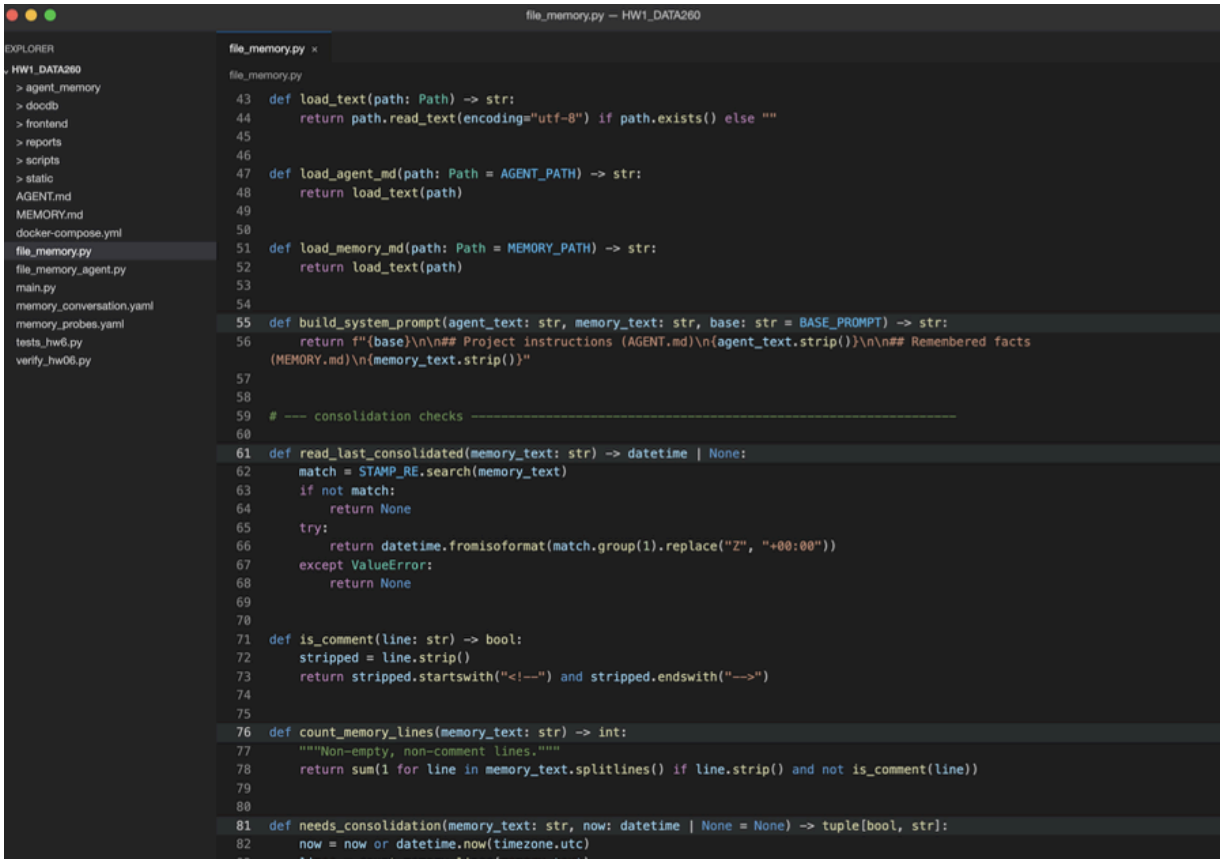


Figure 4.2 - Loading functions, build_system_prompt(), read_last_consolidated(), count_memory_lines(), needs_consolidation().

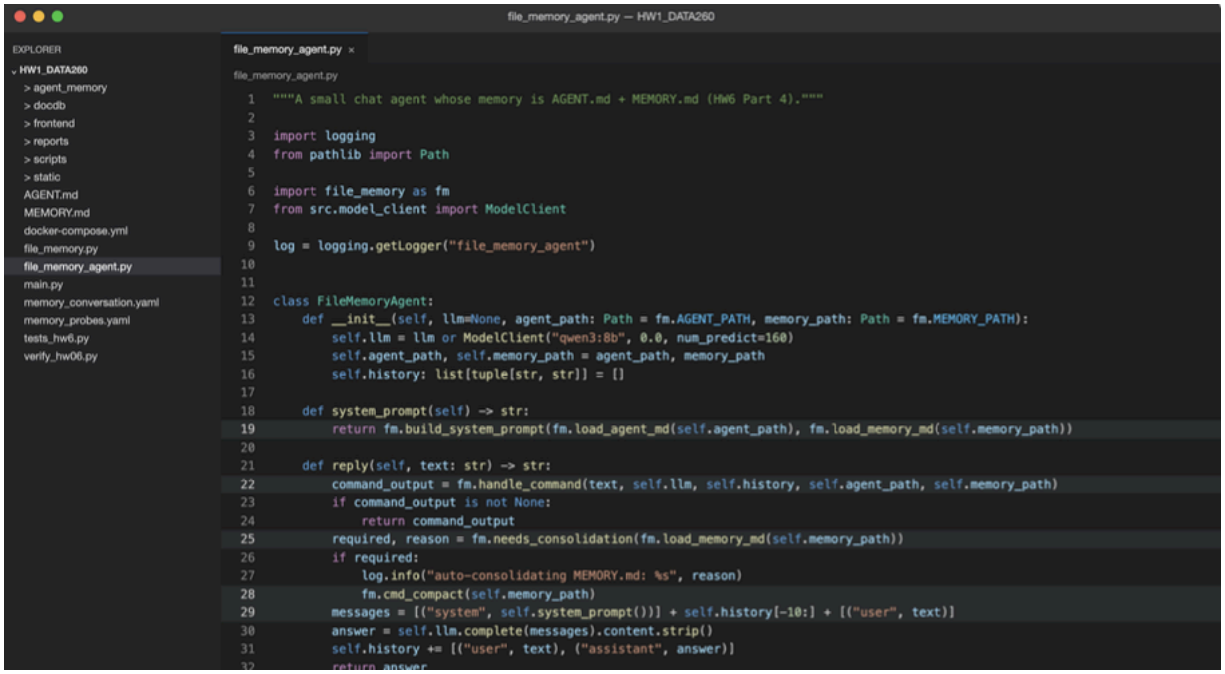


Figure 4.3 - file_memory_agent.py: the system prompt contains AGENT.md and MEMORY.md; slash commands are handled before the model is called.

III. Commands

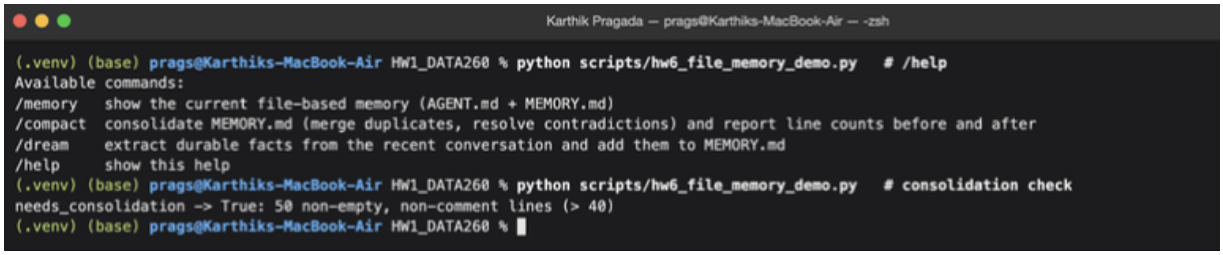


Figure 4.4 - /help output and the consolidation check on the original 55-line MEMORY.md (excerpts of scripts/hw6_file_memory_demo.py).

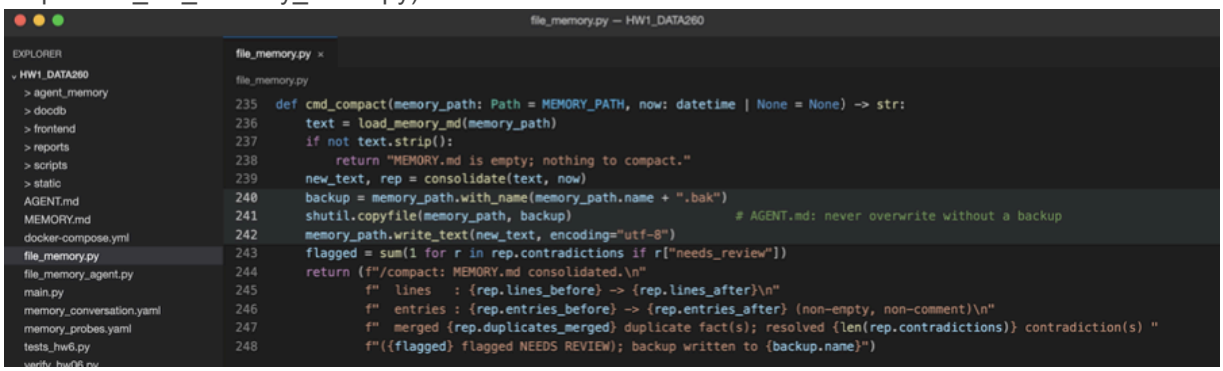


Figure 4.5 - cmd_compact(): writes a backup first (AGENT.md prohibits overwriting MEMORY.md without one) and reports line counts.

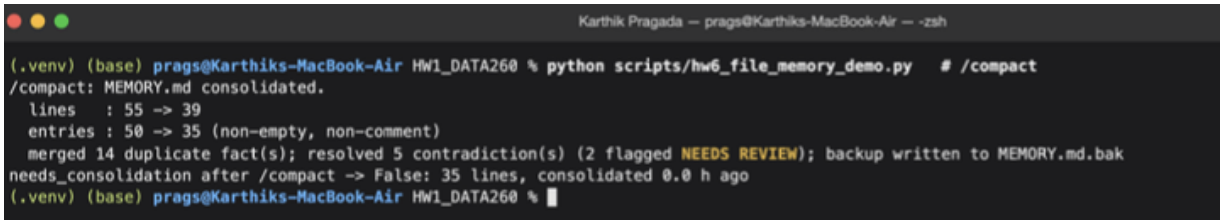


Figure 4.6 - /compact on the 55-line file: lines 55 -> 39, entries 50 -> 35, with the backup file named.

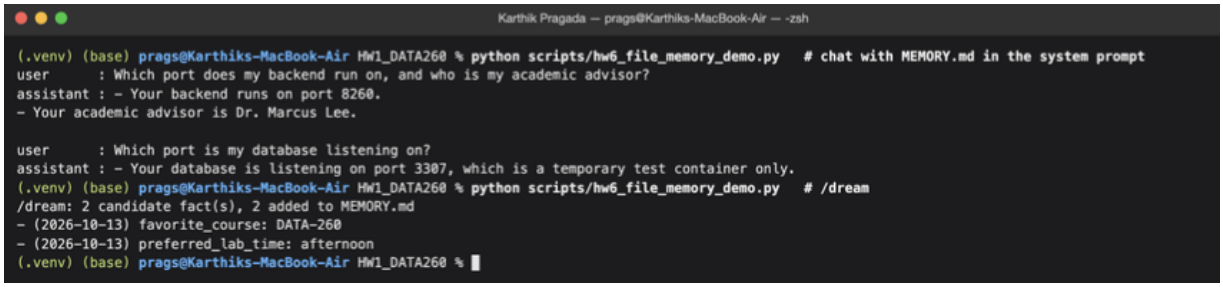


Figure 4.7 - A chat that relies on MEMORY.md in the system prompt, then /dream adding facts learned from the conversation.

IV. A function that follows AGENT.md

memory_health_report() was written under the AGENT.md rules (type hints on every parameter and the return value, 11 lines, snake_case, no print , no eval , an offline assert

based test). scripts/check_agent_md.py verifies those rules from the source with the ast module; 10 of 10 checks pass, and the 13 offline tests pass without a network, database or LLM.

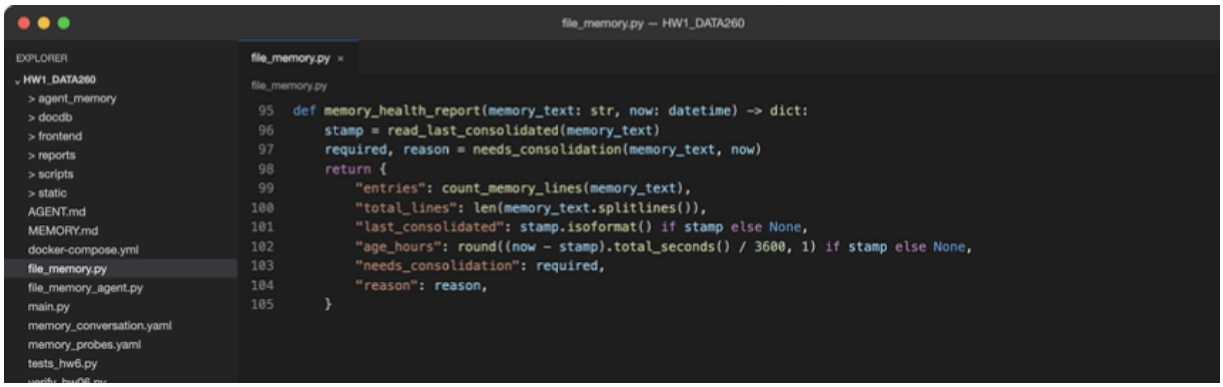


Figure 4.8 - file_memory.memory_health_report().

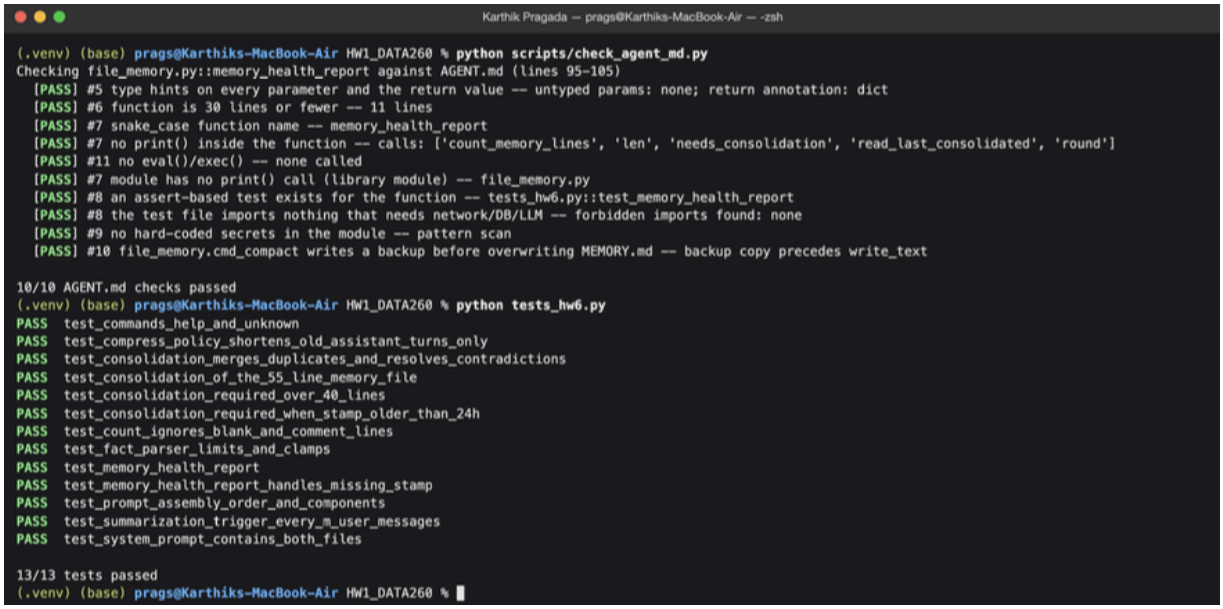


Figure 4.9 - check_agent_md.py (10/10) and tests_hw6.py (13/13).

V. Consolidating a 55-line MEMORY.md

The test file (reports/hw06/raw/memory_consolidation/MEMORY.original.md) has 55 lines with 14 duplicated facts and 5 contradictions. Consolidation merges duplicates (keeping a count), resolves each contradiction with an explicit rule (most recent dated entry wins) and **never drops the older value silently**: it moves it into a "Resolved contradictions" audit section. Entries whose winning value contains qualifier words (temporary, only, test container ...) are flagged NEEDS REVIEW. A second pass leaves the file unchanged.

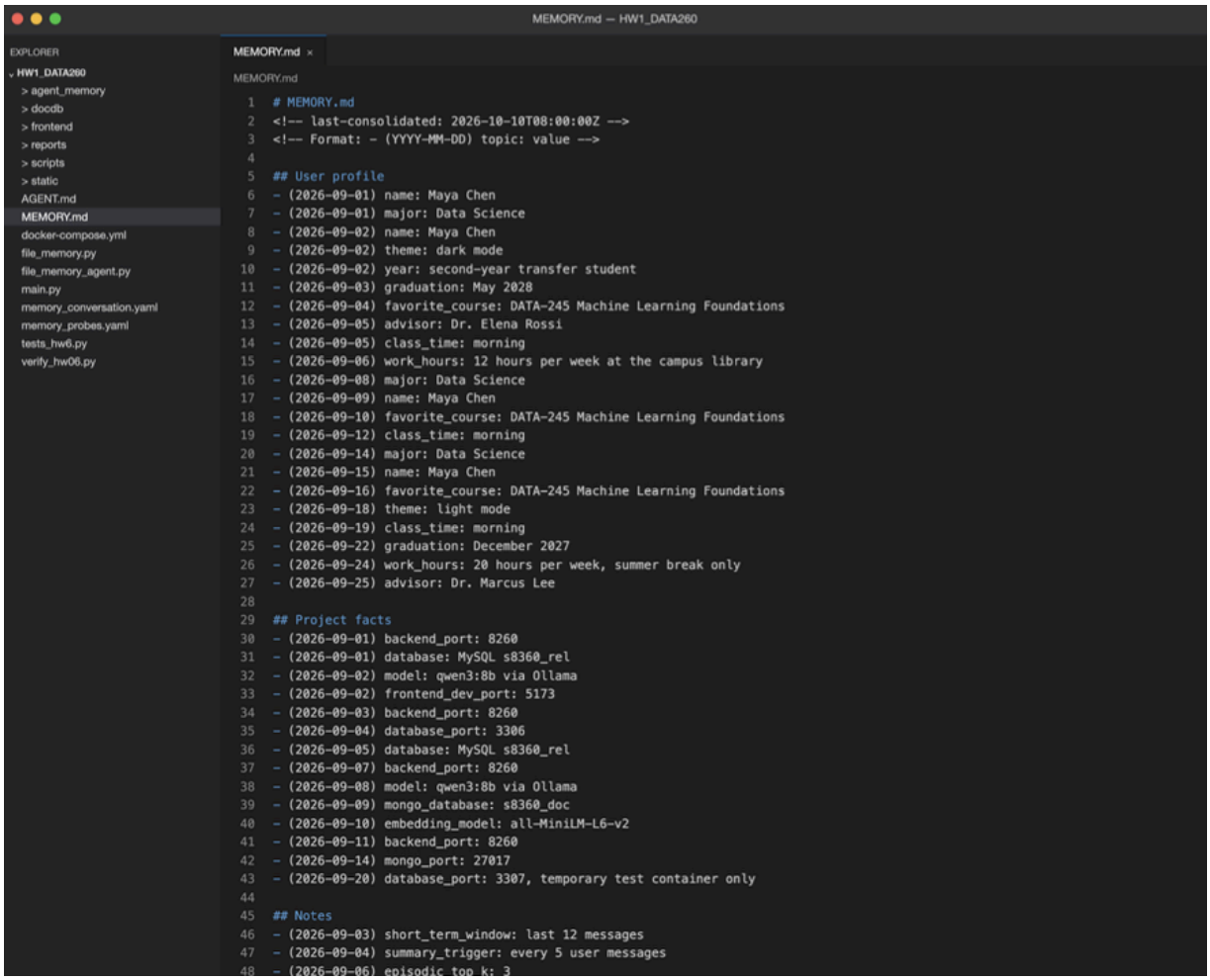


Figure 4.10 - MEMORY.md before consolidation: 55 lines (e.g. backend_port x4, name x4; theme, graduation, advisor, work_hours and database_port contradict).

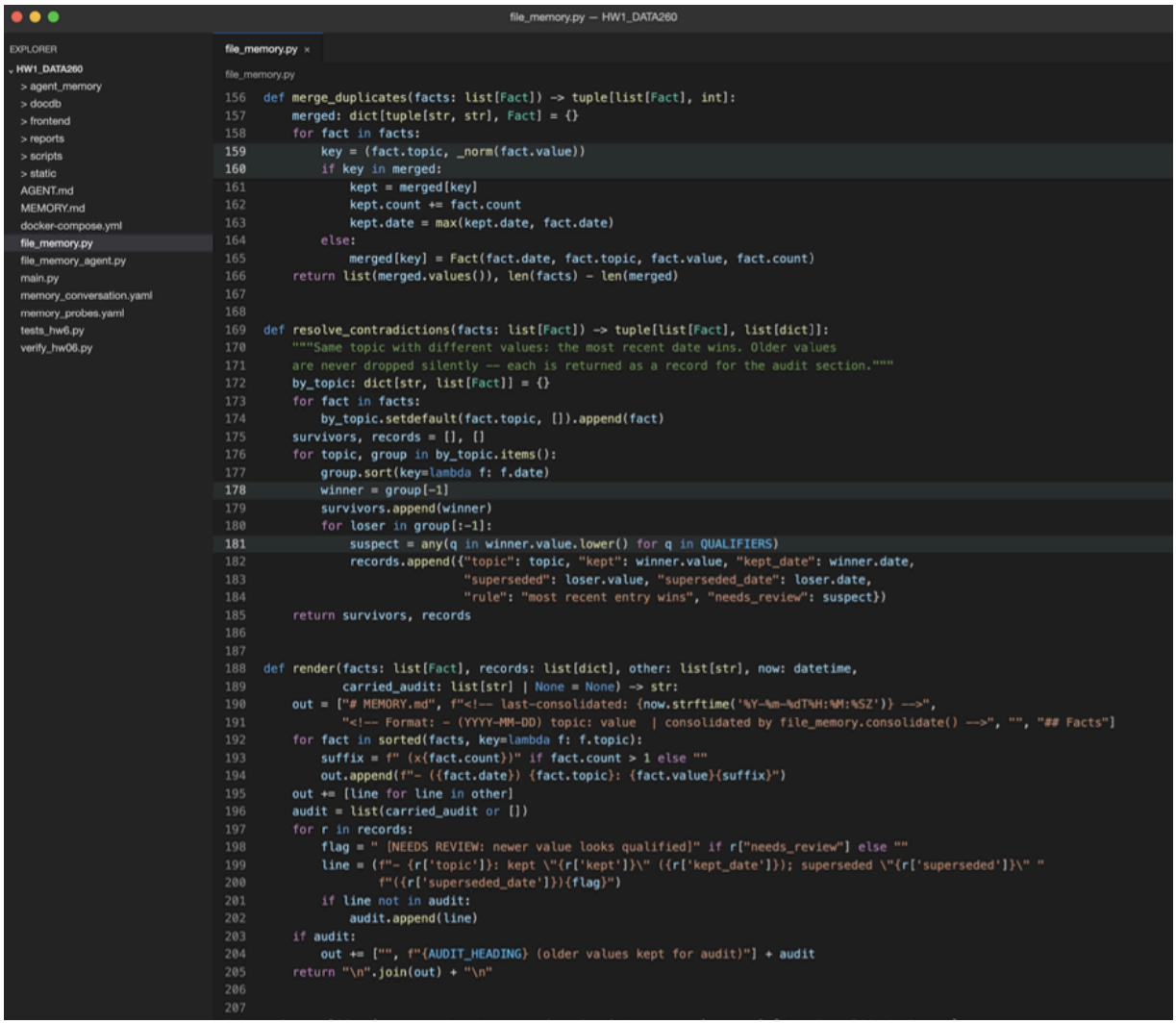


Figure 4.11 - Duplicate merge, contradiction resolution and audit rendering in file_memory.py.

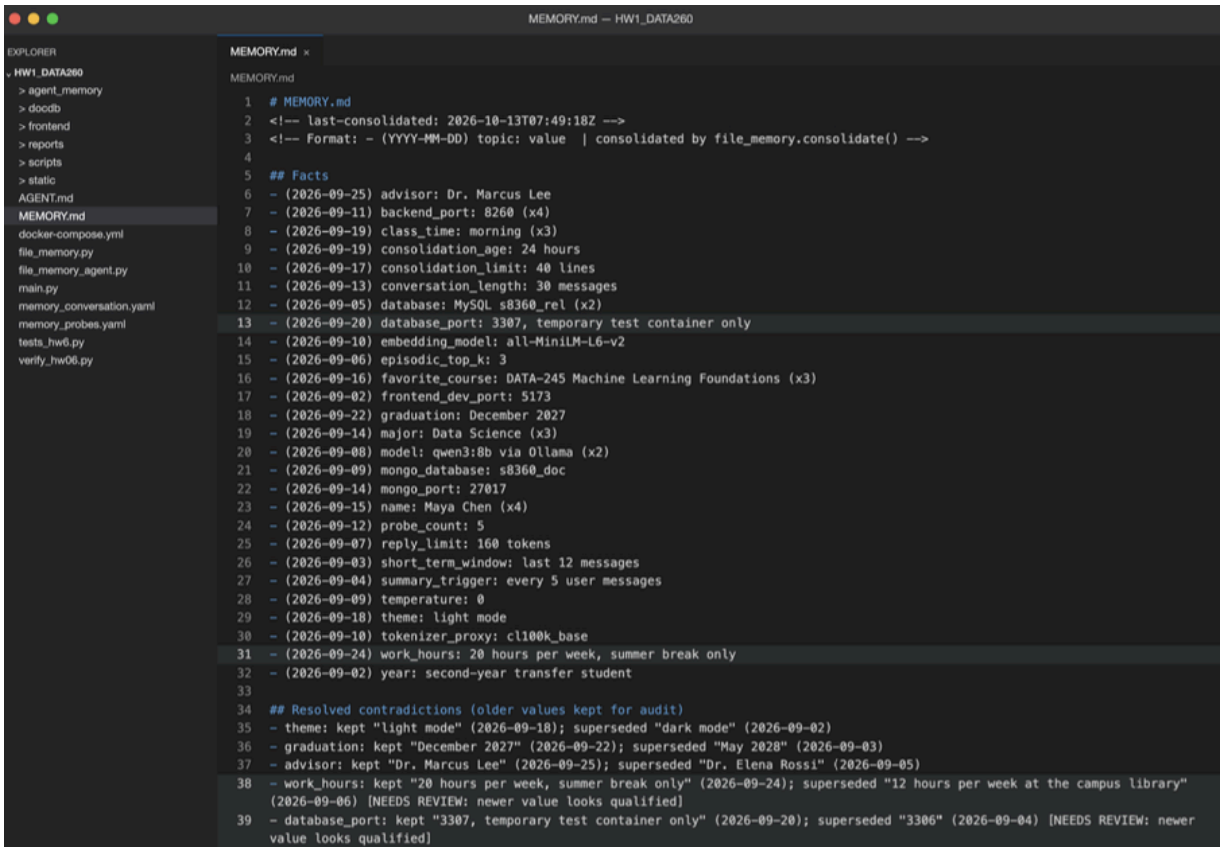


Figure 4.12 - MEMORY.md after consolidation: 39 lines. Highlighted: lines 13 and 31 (database_port, work_hours) and their audit entries 38-39, the two wrongly resolved values.

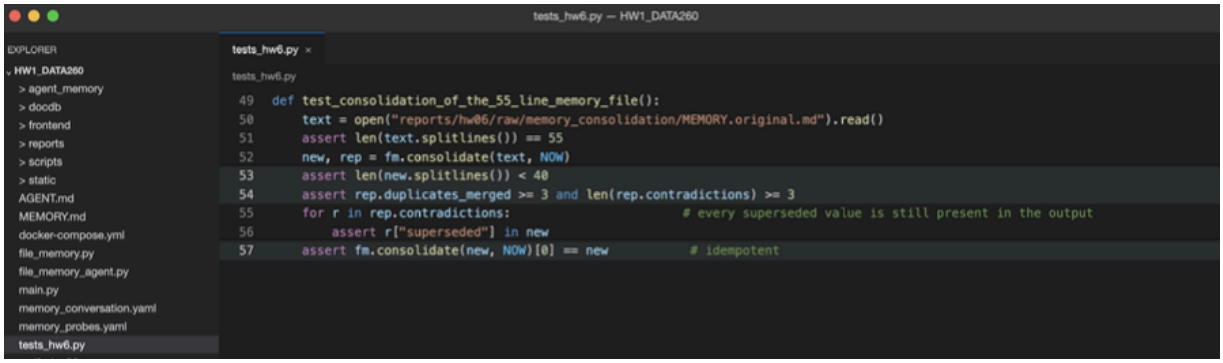


Figure 4.13 - Test asserting 55 -> under 40 lines, every superseded value still present, and idempotence.

VI. Contradictions that were resolved incorrectly

Topic	Kept (newer)	Superseded (older)	What is actually true	Correct?
theme	light mode (09- 18)	dark mode (09-02)	light mode	yes

graduation	December 2027 (09-22)	May 2028 (09-03)	December 2027	yes
advisor	Dr. Marcus Lee (09-25)	Dr. Elena Rossi (09-05)	Dr. Marcus Lee	yes
work_hours	20 hours per week, summer break only (09- 24)	12 hours per week at the campus library (09-06)	12 hours per week (the 20 h figure is a summer-only exception)	**no** - flagged NEEDS REVIEW
database_port	3307, temporary test container only (09-20)	3306 (09-04)	3306 (3307 is a throw-away test container)	**no** - flagged NEEDS REVIEW

"Most recent wins" is wrong exactly when the newer value is a temporary exception rather than a replacement. The consequence is visible: with the consolidated file in the system prompt the model answered that the database listens on port 3307 (Figure 4.7). The qualifier heuristic flags both cases but cannot decide them, so those two lines need a human to restore the older value or mark the newer one as a scoped exception. "Truth" here is what I defined when writing the fixture file.

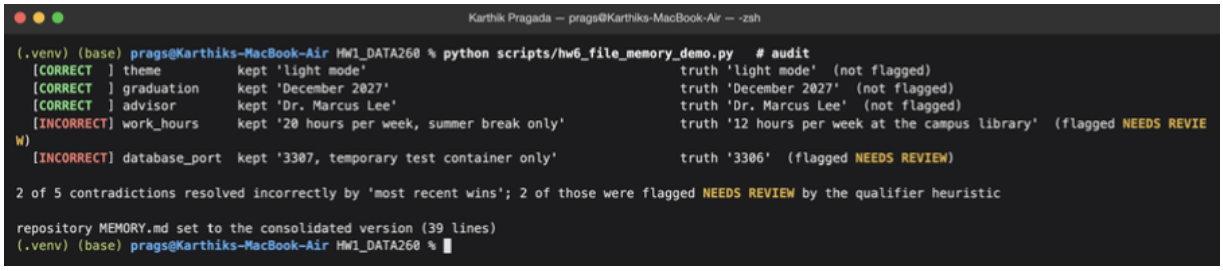


Figure 4.14 - Audit of the five resolutions against the known truth.

VII. Version control

The original 55-line file and the consolidated 39-line file are committed as two separate commits of MEMORY.md , so the change is reviewable as a diff.

Figure (pending) - git log / diff of MEMORY.md (original commit, then consolidated commit). To be captured after the two commits are created; nothing has been committed yet.

VIII. MongoDB memory versus file-based memory

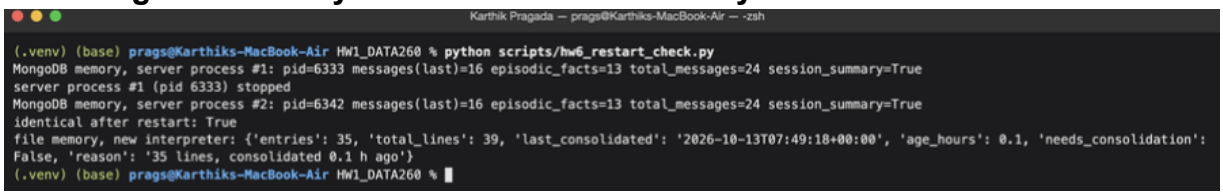


Figure 4.15 - Persistence after a restart: MongoDB-backed memory read by two different server processes, and MEMORY.md reloaded by a new interpreter.

Criterion	MongoDB memory (Part 2)	File-based memory (Part 4)
Inspectability	Needs mongosh / Compass or an API call; many small documents and 384-number vectors	Open in any editor; plain text; changes are a Git diff
Persistence after restart	Survives (identical counts before and after a server restart, see the restart figure above)	Survives (it is a file; reloaded with the same 35 entries); must be committed or backed up to survive a lost disk
Scalability	Grows with users and turns; retrieval keeps the prompt small (top-3 facts) - 85 episode documents in the collection after the experiment, still only 3 in each prompt	Whole file goes in every prompt: AGENT.md 438 + MEMORY.md 737 tokens even when irrelevant; must be capped (40 lines) and consolidated
Must-not forget facts	Can be missed: the fact must be extracted and then ranked in the top-3 by similarity	Always present in the prompt, so a pinned fact cannot be dropped by retrieval; but a wrong consolidation silently changes it
Update path	Automatic (extraction and summaries every turn, costs tokens)	Manual or by /dream and /compact; cheap, deterministic, auditable

Conclusion. The two are complementary. File memory is the right place for a small set of pinned facts and project rules that must always be in the prompt and be reviewable by a

person; MongoDB memory is the right place for the large and growing conversation history, where retrieval keeps the prompt small. The audit in section VI shows the file approach has its own failure mode: consolidation can quietly turn a temporary exception into the "truth", which is why the older value is kept and flagged.